

Case-Project-ITM0

Berber, Nash Andre

Cayman, Florence Anne

De Guzman, Gio Miguelle

Gaspar, Jet Angelo

Game Industry

NoSQL

Traditional database management systems were a product of everyday business operations while NoSQL was a response to the growing needs of web scalability where data often comes as unstructured (Fierro and Chhatrapati, 2021). Because of its flexibility it can handle big-data and unstructured data like emails, text documents, and social media posts (Jonker and Gomstyn, 2025). Since NoSQL was made to address scalability, it handles a larger volume of requests and data at a lower cost when compared to SQL (Bloria, 2024). While databases are not ideal for storing synchronous game state, asynchronous blobs on the other hand can be managed by NoSQL technologies (Hare, 2017).

SQL follows the ACID (Atomicity, Consistency, Isolation, Durability) principle where transactions either happen or rollbacks to the previous state, transactions follow constraints and do not interfere with each other, while commits are stored immediately in hard storage (Lau, 2024). The BASE principle refers to Basically Available, Soft State, and Eventually Consistent. This principle sacrifices some level of consistency to increase performance and availability. Nodes may have different (Soft State) data but will eventually become consistent if no new update occurs. The laxness of the BASE principle allows games to have lower latency by being able to handle more requests and transactions (Lau, 2024).

Before modern database technology, companies and businesses usually had one database server that handled requests from different application clients, they increased performance by increasing the specs of their singular database server (vertical scaling) which was expensive (Fierro and Chhatrapati, 2021). As stated above, NoSQL was a response to the necessity of cheaper scalability. NoSQL enables the use of horizontal scalability, where multiple machines are used to act as a distributed database system. This enables developers to start with small deployment and eventually scale out as the user base grows, without expending too much money on expensive hardware.

With the single server distribution model, you utilize the database in one machine, and since you only have one machine you can only scale vertically. The sharding distribution model is simply about breaking up data sets into multiple pieces in multiple machines, while the replication distribution model is about keeping the same set of data in multiple machines (Brooker, 2023). These two concepts enable NoSQL to be scalable. Users can access the same set of data through different servers (replication), and high traffic sets of data can be separated according to a specific category (sharding). If you utilize `player_id` to shard the database, you could possibly distribute the traffic evenly among the servers that make up your whole database.

Consistency types concern update and read consistency. Update consistency is the resilience of the database to keep its integrity regardless of concurrent updates (write-write conflicts) while read consistency is its resilience when read and write requests (read-write conflicts) happen at the same time (Stearns and Rosenkrantz, 1981). There are two approaches on managing write-write conflicts, by letting conflicts happen and resolving them through detection (optimistic approach), or by preventing conflicts to happen by usually locking read/write access (pessimistic approach) (Adetunji, 2024). There are various types of read consistency: where data items should make sense together (logical), replicas are updated with one another (replication), and making changes presents in related reads (session).

Different NoSQL Technologies

The Key-Value databases keep the data in key-value pairs that are optimized for high-speed lookups and therefore used for caching, session storage, and real-time recommendations. Wide-column databases store their data in rows and columns, this facilitates efficient storage and retrieval of massive amounts of data, it is widely applied in big data and time-series use cases. A document database uses semi-structured data in formats such as JSON or BSON, utilizing this structure makes it a schema-less database well-suited to changing application needs. Graph databases structure the data as nodes and edges or entities and relationships, this allows for quick querying of complex relationships which are widely used in

social networks and fraud detection. In our project, we use Python with MongoDB to create a simple game where users can create accounts, upgrade character stats, fight, view inventory, and shop for in-game items.

Geospatial Data

MongoDB is a powerful NoSQL database that provides built-in support for geospatial data, it allows developers to store and query information that are related to geographical locations. In many applications that include geographical locations such as riding services, delivery services, or location systems, the ability to work with geographical coordinates is essential. MongoDB simplifies this through specific data formats, indexing mechanisms, and query operators.

When representing geographic data, MongoDB supports 2 formats:

1. **GeoJSON document**

- A standard format for encoding geographic structures like Point, LineString, and Polygon.

2. **Legacy coordinate pairs**

- A simpler format using [x, y], usually for flat 2D maps.

MongoDB uses GeoJSON format to store locations. And GeoJSON always uses longitude first, then latitude, which aligns with global mapping standards.

Sample Code:

```
db.campuses.find({
  location: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [120.9506, 15.4619]
      },
      $maxDistance: 2500
    }
  }
})
```

MongoDB vs Relational Databases

a. Structure

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, allowing for dynamic schemas where each document can have a different structure. While in relational databases like MySQL store data in tables with fixed columns, requiring a predefined schema. This flexibility in MongoDB enables developers to store unstructured or semi-structured data without the need for complex schema migrations.

MongoDB and MySQL have a lot in common. Both can store large amounts of data and help retrieve it quickly with indexing. They let you search and work with the data through queries, with MongoDB using its own system and MySQL using SQL. They also both offer security features, like making

sure only authorized users can access the data. These similarities make both MongoDB and MySQL good at managing and working with data in different types of applications.

b. Database Commands

Creating a Database

In MongoDB, there is no specific command to create a database. A database is created implicitly when you switch to it using the use statement. While in MySQL, it uses the CREATE DATABASE statement to create a new database.

Example command for creating database in MongoDB:

use players

Output:

switched to db players

Example command for creating database in MySQL:

CREATE DATABASE players;

Output:

Query OK, 1 row affected

Selecting a Database

In MongoDB, a database is selected by using the use statement, which switches to the specific database. In MySQL, it also uses the USE statement to select or switch to a specific database.

Example command for switching database in MongoDB:

use players

Output:

switched to db players

Example command for switching database in MySQL:

use players;

Output:

Database changed

Creating a Collection

In MongoDB, collections are created using the createCollection statement. In MySQL, tables are created using the CREATE TABLE statement.

Example command for creating a collection in MongoDB:

db.createCollection("studentCollection")

Output:

{ ok: 1 }

Example command for creating a table in MySQL:

```
CREATE TABLE studentTable (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  age INT  
);
```

Output:

Query OK, 0 rows affected

Deleting a Collection

In MongoDB, collections can be deleted using the `sampleCollection.drop()` statement. In MySQL, tables can be deleted using the `DROP TABLE` statement.

Example command for deleting a collection in MongoDB:

```
db.playersCollection.drop()
```

Output:

true

Example command for deleting a table in MySQL:

```
DROP TABLE playersTable;
```

Output:

Query OK, 0 rows affected

Deleting a Database

In MongoDB, a database can be deleted using the `dropDatabase` statement, but you must first switch to the target database using the `use` statement. In MySQL, a database can be deleted using the `DROP DATABASE` statement.

Example command for deleting a database in MongoDB:

```
use players
```

```
db.dropDatabase()
```

Output:

```
{ "dropped" : "players", "ok" : 1 }
```

Example command for deleting a database in MySQL:

```
DROP DATABASE players;
```

Output:

Query OK, 1 rows affected

c. CRUD operations

Create

In MongoDB, documents are inserted into collections using the `insertOne` or `insertMany` methods. In MySQL, rows are added to tables using the `INSERT INTO` statement.

Example command for inserting in MongoDB:

```
db.playerCollection.insertOne(  
  { name: "John", class: "Mage" } )
```

Output:

```
{  
  "acknowledged" : true,  
  "insertedId" : ObjectId("5f50c31f1c4ae10b8d9b3c6a")  
}
```

Example command for inserting in MySQL:

```
INSERT INTO playerTable (name, class)  
VALUES ('John', 'Mage');
```

Output:

Query OK, 1 row affected

Read

In MongoDB, documents can be retrieved using the find() method. In MySQL, data is retrieved from tables using the SELECT statement.

Example command for reading in MongoDB:

```
db.playerCollection.find()
```

Output:

```
{ "_id" : ObjectId("5f50c31f1c4ae10b8d9b3c6a"),  
  "name" : "John",  
  "class" : "mage" }
```

Example command for reading in MySQL:

```
SELECT * FROM playerTable;
```

Output:

i d	name	class
1	John	Mage

Update

In MongoDB, documents are updated using the updateOne or updateMany methods with update operators like \$set. In MySQL, data is updated using the UPDATE statement.

Example command for updating in MongoDB:

```
db.playerCollection.updateOne(  
    {name: "John" }, { $set: {class: "Warrior"}} )
```

Output:

```
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

Example command for updating in MySQL:

```
UPDATE playerTable  
SET class = 'Warrior'  
WHERE name = 'John';
```

Output:

```
Query OK, 1 row affected
```

Delete

In MongoDB, documents are deleted using deleteOne or deleteMany. In MySQL, rows are deleted using the DELETE FROM statement.

Example command for deleting in MongoDB:

```
db.playerCollection.deleteOne(  
    {name: "John" } )
```

Output:

```
{ "acknowledged" : true, "deletedCount" : 1 }
```

Example command for deleting in MySQL:

```
DELETE FROM playerTable  
WHERE name = 'John';
```

Output:

Query OK, 1 row affected

d. Aggregate functions

Count

In MongoDB, `countDocuments()` is used to count the number of documents in a collection. In MySQL, `COUNT()` is used to count the number of rows in a table.

Example command for count in MongoDB:

```
db.playerCollection.countDocuments()
```

Output:

1

Example command for count in MySQL:

```
SELECT COUNT(*) FROM playerTable;
```

Output:

COUNT(*)
1

Average

In MongoDB, the `$avg` operator is used in the aggregate method. In MySQL, `AVG()` calculates the average of numeric values.

Example command for average in MongoDB:

```
db.playerCollection.aggregate([
  {
    $group: {
      _id: null,
      average: {$avg: "$score"}
    }
  }
])
```

Output:

```
{_id: null, "average": 423}
```

Example command for average in MySQL:

```
SELECT AVG(score) FROM playerTable;
```

Output:

AVG(score)
423

Minimum

In MongoDB, the `$min` operator is used with aggregation to find the smallest value. In MySQL, `MIN()` returns the smallest value in a column.

Example command for minimum in MongoDB:

```
db.playerCollection.aggregate([
```

```
{
  $group: {
    _id: null,
    min: {$min: "$score"}
  }
}
```

Output:

```
{_id: ObjectId('67f35e98ef1af5e921d665a2'),
"minGrades" : 200 }
```

Example command for minimum in MySQL:

```
SELECT MIN(score) FROM playerTable;
```

Output:

MIN(score)
200

Maximum

In MongoDB, the \$max operator is used to get the highest value from a document.
In MySQL, MAX() returns the highest value in a column.

Example command for minimum in MongoDB:

```
db.playerCollection.aggregate([
{
  $group: {
    _id: null,
    max: {$max: "$score"}
  }
}]
```

Output:

```
{_id: ObjectId('67f35e98ef1af5e921d665a2'),
"maxGrades" : 700 }
```

Example command for minimum in MySQL:

```
SELECT MAX(score) FROM playerTable;
```

Output:

MAX(grades)
700

e. Connection to MongoDB using Python

To make a connection to MongoDB using Python, we need to install the **pymongo** library.

Importing pymongo library:

```
from pymongo import MongoClient
```

Connection to MongoDB:

```
Client = MongoClient("mongodb://localhost:27017/")
```

Creating and Connecting to a Database:

```
db = client["gameSample"]
```

Creating and Accessing a Collection:

```
account = db["accounts"]
```

Inserting a Document:

```
account.insert_one({"name": "Mario", "score": 100})
```

Finding a Document:

```
account.find_one({"name": "Mario"})
```

Updating a Document:

```
account.update_one({"name": "Mario"}, {"$set": {"name": "Luigi"}})
```

Deleting a Document:

```
account.delete_one({"name": "Luigi"})
```

Dropping a Collection:

```
account.drop()
```

Dropping a Database:

```
client.drop_database("gameSample")
```

5. Game Sample (Using Love2D and Flask)

Full Code Base: https://github.com/Voided-Name/menu_test

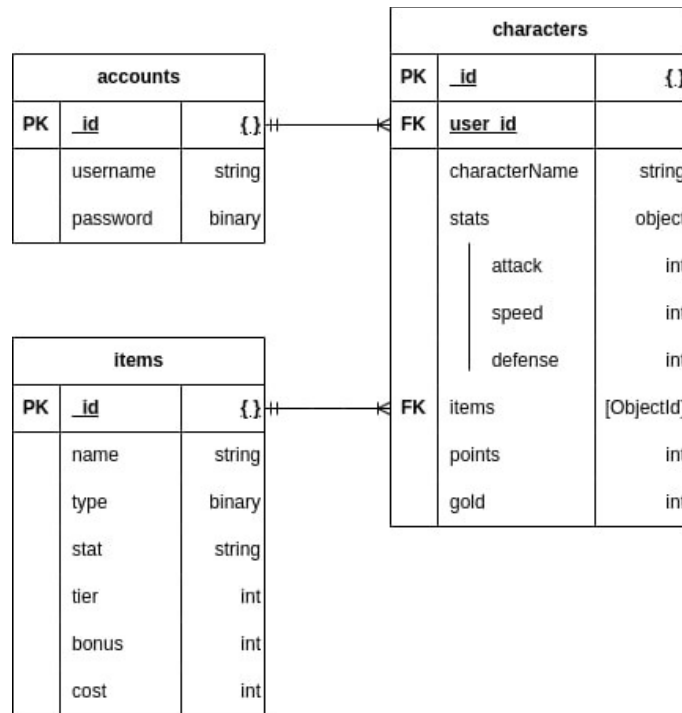


Figure 1. Database Design

Initial Setup

```

test> use game
switched to db game
game> db.createCollection("accounts")
{ ok: 1 }
game> db.createCollection("characters")
{ ok: 1 }
game> db.createCollection("items")
{ ok: 1 }
  
```

Figure 2. Inserting Three Collections

```

game> db.getCollectionNames()
[ 'characters', 'accounts', 'items' ]
  
```

Figure 3. Three Collections

```

game> db.accounts.insertMany([
... {
...   username: 'nash',
...   password: new BinData(0, 'JDDJiJDEyJE1hDXg0Umthd3hML3FxNhkuc lRXS3V4T1BFZGdzSjAud@VYSmJNNy5DemxFT lE0NUFzek tp ')
... },
... {
...   username: 'jet',
...   password: new BinData(0, 'JDDJiJDEyJE9XU2g0b jkwb313Mm5HVS5mSz l3TSSwMVL1T01yVHRaNGF3Q2VPMXN0Z3pZuU9qVmh4aD lX ')
... },
... {
...   username: 'test',
...   password: new BinData(0, 'JDDJiJDEyJDBVwDB1Q2JHn0x1Y1NhSFFUS0pPnmVGaVd tb lVTa1BndUR4VTFNY2RqZ0VqVUo0w lJBNT lU ')
... },
... {
...   username: 'gio',
...   password: new BinData(0, 'JDDJiJDEyJEHzNlphTEg2M l dqV1ZkYSSyZ2dJaE9 lMmuvVnNpdUpvYnZ6RVNzS1VobnZ3RFB l c jZodGNt ')
... },
... {
...   username: 'GamerJob',
...   password: new BinData(0, 'JDDJiJDEyJDUuM2p i cHV0U3pzR0NM lMkZnpsdGVTNFJ lU3hXODhGZ2tr dUL5OEQxN3NHdVkyQ lZwd2Vh ')
... },
... {
...   username: 'raisen',
...   password: new BinData(0, 'JDDJiJDEyJHh lMk lQW lVsZ jZrMVR iTS9J lVldvdE9ZM lJyZ1AuSXdhaGVoQm5sZGRKa jhwaGht0VV4amFx ')
... },
... {
...   username: 'flint',
...   password: new BinData(0, 'JDDJiJDEyJFZ6Lk1rM jhZL2FzUEN3clWZ0dGRPV0h1azBSck l zRm l0UEVqR l lZcXp5S6xdM25kNkxldZT l z ')
... },
... {
...   username: 'luna',
...   password: new BinData(0, 'JDDJiJDEyJE8zn lJ6ZVVSQy9NVExocG8vW lE4Z3o2bHVaUG l aTzZDVFJ5QU lTYmR30G9Ob lB6ekRN a3 l x ')
... },
... {
...   username: 'kai',
...   password: new BinData(0, 'JDDJiJDEyJG9FbFZ0WnR0dzN0dU9YyZf lVkodKT3Y l dTFadVV2aFBmUFJ zUGFBTVg5V0 lK lH lRON3dxbmJu ')
... },
... {
...   username: 'val',
...   password: new BinData(0, 'JDDJiJDEyJHZyb0V lVVExLzVYQzhTT0 lJbW lSSE lXdzR4T3VzbU lFMVXZ2UmpZcFVYaGh0aGV5V3k30GdZ ')
... }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6817a063c7cd9bc1ce0133b8'),
    '1': ObjectId('6817a063c7cd9bc1ce0133b9'),
    '2': ObjectId('6817a063c7cd9bc1ce0133ba'),
    '3': ObjectId('6817a063c7cd9bc1ce0133bb'),
    '4': ObjectId('6817a063c7cd9bc1ce0133bc'),
    '5': ObjectId('6817a063c7cd9bc1ce0133bd'),
    '6': ObjectId('6817a063c7cd9bc1ce0133be'),
    '7': ObjectId('6817a063c7cd9bc1ce0133bf'),
    '8': ObjectId('6817a063c7cd9bc1ce0133c0'),
    '9': ObjectId('6817a063c7cd9bc1ce0133c1')
  }
}
game>

```

Figure 4. Inserting 10 Accounts

```

game> db.accounts.find()
[
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133b8'),
    username: 'nash',
    password: Binary.createFromBase64('JDJiJDEyJE1hOXg0Umthd3hML3FxdHkwc lRXS3V4T1BFZGdzSjAwd0VYSmJNNy5DemxFTlE0NlFzektp', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133b9'),
    username: 'jet',
    password: Binary.createFromBase64('JDJiJDEyJE9XU2g0b jkwb313MmSHVSSmSz l3TS5wMV l1T0IyVHRaNGF3Q2VPMXN0Z3pZlU9qVmh4eDlX', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133ba'),
    username: 'test',
    password: Binary.createFromBase64('JDJiJDEyJDBVWDB1Q2JHN0x1Y1NhSFFUS0pPNmVGaVdtb lVTa1BndUR4VTFNY2RqZ0VqVUo0WlJBNT lU', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bb'),
    username: 'gio',
    password: Binary.createFromBase64('JDJiJDEyJEMzNlphTEg2MldqV1ZkYSSyZ2dJaE9IMmwvVnNpdUpvYnZ6RVNzS1VobnZ3RFB1c jZodGNt', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bc'),
    username: 'GamerJob',
    password: Binary.createFromBase64('JDJiJDEyJDVuM2picHV0U3pzR0NXMlMxZnpsdGVTNFJ1U3hXODhGZ2trdUUSOEQxN3NHdVkyQ lZld2Vh', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bd'),
    username: 'naisen',
    password: Binary.createFromBase64('JDJiJDEyJHh lMkJQWlVsZ jZrMVR lTS9JVldvdE9ZMlJyZ1AuSXdhAGVoQm5sZGRKa jhwaGht0VV4amFx', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133be'),
    username: 'flint',
    password: Binary.createFromBase64('JDJiJDEyJFZ6Lk1rM jhZl2FzUEN3cWZOdGRPV0h1azBSck lZRm l0UEVqR l lZcXp5SGxMM25kNkxMZT lZ', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bf'),
    username: 'luna',
    password: Binary.createFromBase64('JDJiJDEyJE8zN1J6ZVV5Qy9NVExocG8vW lE4Z3o2bHVaU61aTzZDVfJ5QU1TYmR3OG9Ob1B6ekRNa3lX', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133c0'),
    username: 'kai',
    password: Binary.createFromBase64('JDJiJDEyJG9FbFZolWnR0dzN0dU9YYzFtVkkxKT3Y1dTFadVV2aFBmUFJzUGFBTVg5V01K1H-RQN3dxbmJu', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133c1'),
    username: 'val',
    password: Binary.createFromBase64('JDJiJDEyJHZyb0V lVVExLzVYQzhTT0 lJbW l5SEtXdzR4T3VzbUFMVXZ2UnpZcFVYaGh0aGV5V3k3OGdZ', 0)
  }
]

```

Figure 5. 10 Accounts

Figure 6. Inserting 10 Characters

```

game> db.characters.find()
[
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133cc'),
    user_id: ObjectId('6817830339595684e701337b'),
    characterName: 'nash1',
    stats: { attack: 5, speed: 2, defense: 3 },
    items: [],
    points: 100,
    gold: 50
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133cd'),
    user_id: ObjectId('6817830339595684e701337b'),
    characterName: 'nash2',
    stats: { attack: 7, speed: 1, defense: 2 },
    items: [],
    points: 110,
    gold: 60
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133ce'),
    user_id: ObjectId('6817830339595684e701337b'),
    characterName: 'nash3',
    stats: { attack: 6, speed: 3, defense: 1 },
    items: [],
    points: 90,
    gold: 45
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133cf'),
    user_id: ObjectId('6817830339595684e701337c'),
    characterName: 'jet1',
    stats: { attack: 4, speed: 5, defense: 2 },
    items: [],
    points: 130,
    gold: 55
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133d0'),
    user_id: ObjectId('6817830339595684e701337c'),
    characterName: 'jet2',
    stats: { attack: 6, speed: 2, defense: 2 },
    items: [],
    points: 95,
    gold: 50
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133d1'),
    user_id: ObjectId('6817830339595684e701337c'),
    characterName: 'jet3',
    stats: { attack: 5, speed: 3, defense: 2 },
    items: [],
    points: 85,
    gold: 40
  },
],

```

```

[
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133d2'),
    user_id: ObjectId('6817830339595684e701337d'),
    characterName: 'test1',
    stats: { attack: 3, speed: 6, defense: 3 },
    items: [],
    points: 100,
    gold: 60
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133d3'),
    user_id: ObjectId('6817830339595684e701337d'),
    characterName: 'test2',
    stats: { attack: 4, speed: 4, defense: 4 },
    items: [],
    points: 120,
    gold: 70
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133d4'),
    user_id: ObjectId('6817830339595684e701337d'),
    characterName: 'test3',
    stats: { attack: 2, speed: 5, defense: 5 },
    items: [],
    points: 110,
    gold: 65
  },
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133d5'),
    user_id: ObjectId('6817830339595684e701337e'),
    characterName: 'gio1',
    stats: { attack: 6, speed: 2, defense: 4 },
    items: [],
    points: 115,
    gold: 80
  },
],

```

Figure 7. 10 Characters

```

game> db.items.insertMany([
...   {
...     name: 'Iron Blade',
...     type: 'weapon',
...     stat: 'atk',
...     tier: 1,
...     bonus: 5,
...     cost: 100
...   },
...   {
...     name: 'Steel Sword',
...     type: 'weapon',
...     stat: 'atk',
...     tier: 2,
...     bonus: 10,
...     cost: 250
...   },
...   {
...     name: 'Flame Sabre',
...     type: 'weapon',
...     stat: 'atk',
...     tier: 3,
...     bonus: 20,
...     cost: 500
...   },
...   {
...     name: 'Light Shoes',
...     type: 'armor',
...     stat: 'spd',
...     tier: 1,
...     bonus: 3,
...     cost: 80
...   },
...   {
...     name: 'Swift Boots',
...     type: 'armor',
...     stat: 'spd',
...     tier: 2,
...     bonus: 6,
...     cost: 200
...   },
...   {
...     name: 'Phantom Greaves',
...     type: 'armor',
...     stat: 'spd',
...     tier: 3,
...     bonus: 12,
...     cost: 450
...   },
...   {
...     name: 'Leather Armor',
...     type: 'armor',
...     stat: 'def',
...     tier: 1,
...     bonus: 4,
...     cost: 120
...   },
... ],

```

```

...   {
...     name: 'Chainmail',
...     type: 'armor',
...     stat: 'def',
...     tier: 2,
...     bonus: 8,
...     cost: 300
...   },
...   {
...     name: 'Dragon Plate',
...     type: 'armor',
...     stat: 'def',
...     tier: 3,
...     bonus: 16,
...     cost: 600
...   },
...   {
...     name: 'Minor Health Potion',
...     type: 'potion',
...     stat: 'hp',
...     tier: 1,
...     bonus: 5,
...     cost: 50
...   },
...   {
...     name: 'Major Health Potion',
...     type: 'potion',
...     stat: 'hp',
...     tier: 2,
...     bonus: 10,
...     cost: 120
...   },
...   {
...     name: 'Elixir of Vitality',
...     type: 'potion',
...     stat: 'hp',
...     tier: 3,
...     bonus: 20,
...     cost: 250
...   }
... ])
...
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6817a308c7cd9bc1ce0133d6'),
    '1': ObjectId('6817a308c7cd9bc1ce0133d7'),
    '2': ObjectId('6817a308c7cd9bc1ce0133d8'),
    '3': ObjectId('6817a308c7cd9bc1ce0133d9'),
    '4': ObjectId('6817a308c7cd9bc1ce0133da'),
    '5': ObjectId('6817a308c7cd9bc1ce0133db'),
    '6': ObjectId('6817a308c7cd9bc1ce0133dc'),
    '7': ObjectId('6817a308c7cd9bc1ce0133dd'),
    '8': ObjectId('6817a308c7cd9bc1ce0133de'),
    '9': ObjectId('6817a308c7cd9bc1ce0133df'),
    '10': ObjectId('6817a308c7cd9bc1ce0133e0'),
    '11': ObjectId('6817a308c7cd9bc1ce0133e1')
  }
}
game>

```

Figure 8. Inserting All Items

```
game> db.items.find()
[
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d6'),
    name: 'Iron Blade',
    type: 'weapon',
    stat: 'atk',
    tier: 1,
    bonus: 5,
    cost: 100
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d7'),
    name: 'Steel Sword',
    type: 'weapon',
    stat: 'atk',
    tier: 2,
    bonus: 10,
    cost: 250
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d8'),
    name: 'Flame Sabre',
    type: 'weapon',
    stat: 'atk',
    tier: 3,
    bonus: 20,
    cost: 500
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d9'),
    name: 'Light Shoes',
    type: 'armor',
    stat: 'spd',
    tier: 1,
    bonus: 3,
    cost: 80
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133da'),
    name: 'Swift Boots',
    type: 'armor',
    stat: 'spd',
    tier: 2,
    bonus: 6,
    cost: 200
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133db'),
    name: 'Phantom Greaves',
    type: 'armor',
    stat: 'spd',
    tier: 3,
    bonus: 12,
    cost: 450
  },
]
```

```
{
  _id: ObjectId('6817a308c7cd9bc1ce0133dc'),
  name: 'Leather Armor',
  type: 'armor',
  stat: 'def',
  tier: 1,
  bonus: 4,
  cost: 120
},
{
  _id: ObjectId('6817a308c7cd9bc1ce0133dd'),
  name: 'Chainmail',
  type: 'armor',
  stat: 'def',
  tier: 2,
  bonus: 8,
  cost: 300
},
{
  _id: ObjectId('6817a308c7cd9bc1ce0133de'),
  name: 'Dragon Plate',
  type: 'armor',
  stat: 'def',
  tier: 3,
  bonus: 16,
  cost: 600
},
{
  _id: ObjectId('6817a308c7cd9bc1ce0133df'),
  name: 'Minor Health Potion',
  type: 'potion',
  stat: 'hp',
  tier: 1,
  bonus: 5,
  cost: 50
},
{
  _id: ObjectId('6817a308c7cd9bc1ce0133e0'),
  name: 'Major Health Potion',
  type: 'potion',
  stat: 'hp',
  tier: 2,
  bonus: 10,
  cost: 120
},
{
  _id: ObjectId('6817a308c7cd9bc1ce0133e1'),
  name: 'Elixir of Vitality',
  type: 'potion',
  stat: 'hp',
  tier: 3,
  bonus: 20,
  cost: 250
}
]
game> █
```

Figure 9. All Items

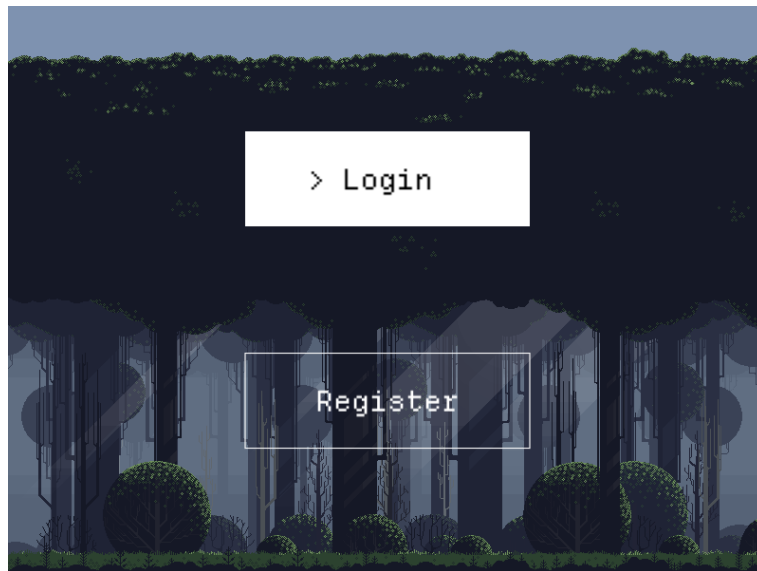
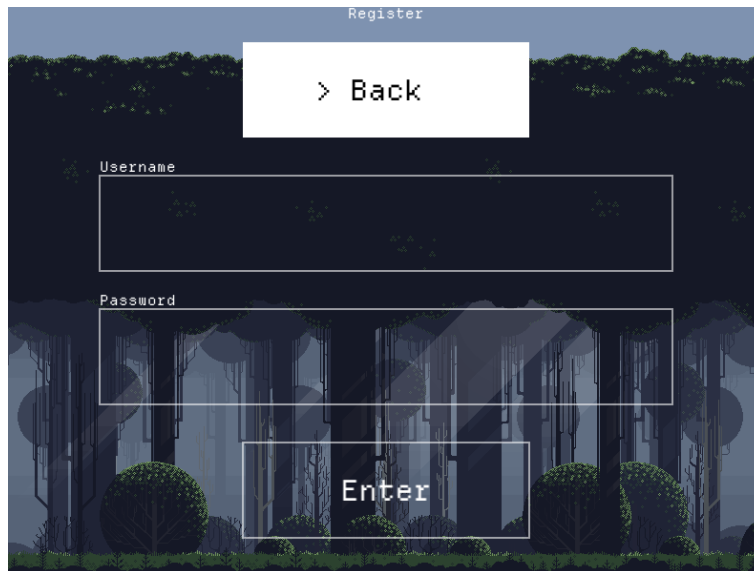


Figure 10. Main Menu

– managing inputs

```
function state.keypressed(key, scancode, isrepeat, switchState, user_id)
  if scancode == "up" or scancode == "down" then -- move down
    if loginButton.selected == false then
      loginButton.selected = true
      registerButton.selected = false
      config.sounds.minimal[1]:clone():play()
    else
      loginButton.selected = false
      registerButton.selected = true
      config.sounds.minimal[2]:clone():play()
    end
  elseif scancode == "return" or scancode == "space" then
    if registerButton.selected == true then
      switchState("register")
      config.sounds.enter:clone():play()
    elseif loginButton.selected then
      switchState("login")
      config.sounds.enter:clone():play()
    end
  end
end
```



Register

> Back

Username

Password

Enter

Figure 11. Register

– backend Code

```
@app.route('/register', methods = ['POST'])
def register():
    if request.method == 'POST':
        data = request.form
        username= data.get('username')
        password = data.get('password', '')

        password_hashed = bcrypt.hashpw(password.encode(),bcrypt.gensalt())

        if account.find_one({"username" : username}):
            return "exists", 200

        # start transaction
        result = account.insert_one({"username": username, "password": password_hashed})

        print(result)
        return "success", 200
```

The image shows a web form titled "Login" centered at the top. Below the title is a white button with the text "> Back". Underneath are two input fields: the first is labeled "Username" and the second is labeled "Password". At the bottom of the form is a white button with the text "Enter". The entire form is overlaid on a dark, stylized background of a forest with tall trees and some green bushes at the bottom.

Figure 12. Login

– backend code

```
@app.route('/login', methods = ['POST'])
def login():
    if request.method == 'POST':
        data = request.form
        username = data.get('username')
        password = data.get('password', '')

        user = account.find_one({"username": username})

        print(user)

        if user and bcrypt.checkpw(password.encode(), user['password']):
            user_id = str(user['_id'])
            return jsonify({"status": "success", "user_id": user_id}), 200

    return "failed", 401-
```



Figure 13. Character Select

– backend code

```
@app.route('/checkChars', methods = ['POST'])
def checkChars():
    if request.method == 'POST':
        data = request.form
        _id = data.get('_id')

        characterCount = characters.count_documents({"user_id": ObjectId(_id)})

        if characterCount > 0:
            return jsonify ({"characterCount": characterCount}), 200
        else:
            return "noChar", 201
    return "failed", 401-
```

```

@app.route('/getChars', methods = ['POST'])
def getChars():
    if request.method == 'POST':
        data = request.get_json()
        print(data)
        _id = data.get('_id')

        allUserChars = characters.find({"user_id": ObjectId(_id)})

        result = []

        for character in allUserChars:
            character['_id'] = str(character['_id'])
            character['user_id'] = str(character['user_id'])

            if isinstance(character.get("items"), list):
                character["items"] = [str(item) for item in character["items"]]

            result.append(character)
        print(result)

        return jsonify(result), 200
    return jsonify({"status": "failed"}), 401-

```

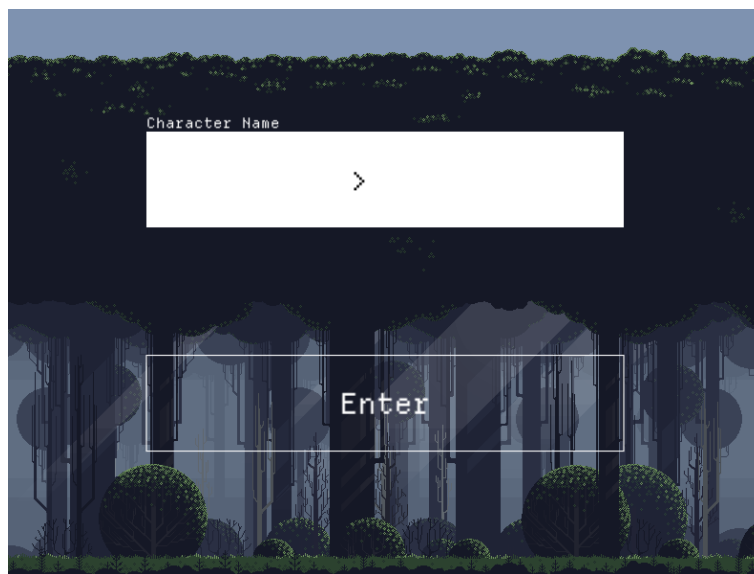


Figure 14. Create Character

– backend code

```

@app.route('/addChar', methods = ['POST'])
def addChar():
    if request.method == 'POST':
        data = request.form
        _id = data.get('_id')
        charname = data.get('charname')

        insertedCharacter = characters.insert_one({"user_id": ObjectId(_id), "characterName": charname, "stats": {"attack": 1, "speed": 1, "defense": 1, "items": {}, "points": 0, "gold": 0}})

        print(insertedCharacter)

        if insertedCharacter:
            return "success", 200

    return "failed", 401-

```

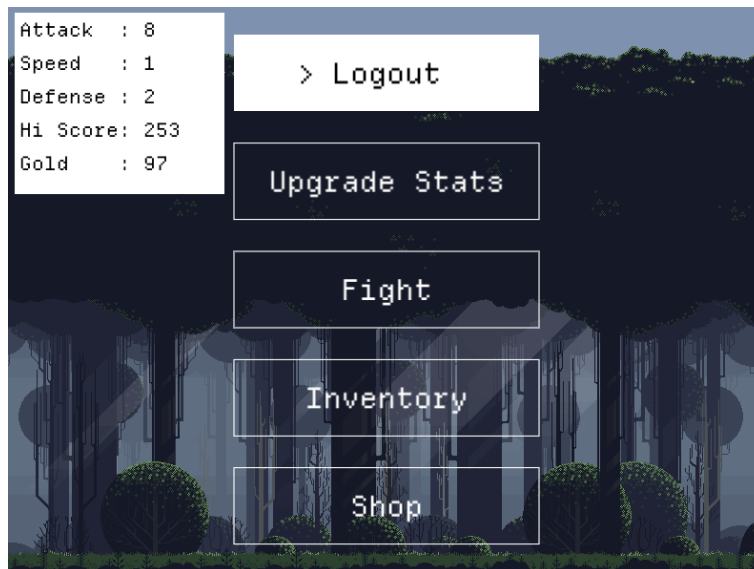


Figure 15. Play Menu



Figure 16. Training

– backend code

```
@app.route('/modifyStat', methods = ['POST'])
def modifyStat():
    if request.method == 'POST':
        data = request.get_json()
        _id = data.get('_id')
        char_id = data.get('char_id')
        stat = data.get('stat')
        points = data.get('points')

        characters.update_one( { "_id": ObjectId(char_id) }, {"$inc": { "stats." + stat: points } })

        return jsonify({"status": "success"}), 200

    return jsonify({"status": "failed"}), 401-
```



Figure 16. Main Gameplay Loop

– backend code

```
@app.route('/setHighScore', methods = ['POST'])
def setHighScore():
    if request.method == 'POST':
        data = request.get_json()

        print("Raw data:", request.data)
        print("Parsed JSON:", data)

        char_id = data.get("char_id")
        score = data.get("score")

        char_score = characters.find_one({"_id": ObjectId(char_id)}, {"_id": 0, "points": 1})
        print(char_id)
        print(char_score)

        if not char_score or char_score['points'] < score:
            characters.update_one({"_id": ObjectId(char_id)}, {"$set": {"points": score}})

        return jsonify({"status": "success"}), 200

    return jsonify({"status": "failed"}), 401-

@app.route('/addGold', methods = ['POST'])
def addGold():
    if request.method == 'POST':
        data = request.get_json()
        _id = data.get('_id')
        gold = data.get('gold')

        characters.update_one({"_id": ObjectId(_id)}, {"$inc": {"gold": gold}})

        return jsonify({"status": "success"}), 200

    return jsonify({"status": "failed"}), 401-
```



Figure 17. Items

– backend code

```
@app.route('/getItems', methods=['POST'])
def getItems():
    data = request.get_json()
    char_id = data.get('char_id')
    owned_item_ids = []

    character = characters.find_one({"_id": ObjectId(char_id)})

    if character:
        owned_item_ids = character.get('items', [])

    # Make sure item IDs are ObjectId
    owned_item_ids = [ObjectId(item_id) for item_id in owned_item_ids]

    owned_items = items.find({"_id": {"$in": owned_item_ids}})

    result = []
    for item in owned_items:
        result.append({
            '_id': str(item['_id']),
            'name': str(item['name']),
            'type': str(item['type']),
            'stat': str(item['stat']),
            'tier': str(item['tier']),
            'bonus': str(item['bonus']),
            'cost': str(item['cost'])
        })

    return jsonify(result), 200
```



Figure 18. Shop

– backend code

```
@app.route('/getShopItems', methods=['POST'])
def getShopItems():
    data = request.get_json()
    char_id = data.get('char_id')
    owned_item_ids = []

    character = characters.find_one({"_id": ObjectId(char_id)})
    if character:
        owned_item_ids = character.get('items', [])

    # Convert all to ObjectId
    owned_item_ids = [ObjectId(item_id) for item_id in owned_item_ids]

    # Find items NOT in the owned list
    shopItems = items.find({"_id": {"$nin": owned_item_ids}})

    result = []
    for shopItem in shopItems:
        shopItem['_id'] = str(shopItem['_id'])
        shopItem['name'] = str(shopItem['name'])
        shopItem['type'] = str(shopItem['type'])
        shopItem['stat'] = str(shopItem['stat'])
        shopItem['tier'] = str(shopItem['tier'])
        shopItem['bonus'] = str(shopItem['bonus'])
        shopItem['cost'] = str(shopItem['cost'])
        result.append(shopItem)

    return jsonify(result), 200
```

```

@app.route('/reduceGold', methods = ['POST'])
def reduceGold():
    if request.method == 'POST':
        data = request.get_json()

        print("Data: ")
        print(data)

        _id = data.get('_id')
        gold = data.get('gold')

        gold = -1 * int(gold)

        characters.update_one({ "_id": ObjectId(_id) }, {"$inc": { "gold" : gold} })

        return jsonify({"status": "success"}), 200

    return jsonify({"status": "failed"}), 401-

@app.route('/addItem', methods = ['POST'])
def addItem():
    if request.method == 'POST':
        data = request.get_json()
        _id = data.get('_id')
        itemId = data.get('itemId')

        characters.update_one({ "_id": ObjectId(_id) }, {"$addToSet": { "items": ObjectId(itemId) }})

        return jsonify({"status": "success"}), 200

    return jsonify({"status": "failed"}), 401-

```

Using Variations of Find ({parameter})

```

game> db.accounts.find({username: "nash"})
[
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133b8'),
    username: 'nash',
    password: Binary.createFromBase64('JDDjiJDEyJE1hOXg0Umthd3hML3FxNHkwcIRXS3V4T1BFZGdzSJAwD0VYSmJNNy5DemxFTIE0NUFzektp', 0)
  }
]

```

```

game> db.accounts.find()
[
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133b8'),
    username: 'nash',
    password: Binary.createFromBase64('JDJiJDEyJE1hOXg0Umthd3hML3FxNHkwc lRXS3V4T1BFZ6dzS jAwd0VYSmJNNy5DemxFT lE0NUFzektp', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133b9'),
    username: 'jet',
    password: Binary.createFromBase64('JDJiJDEyJE9XU2g0b jkwb3l3Mm5HVS5mSz l3TS5wMV l1T01yVHRaNGF3Q2VPMXN0z3pZwU9qVmh4eDlX', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133ba'),
    username: 'tet',
    password: Binary.createFromBase64('JDJiJDEyJDBVWDB1Q2JHN0x1Y1NhSFFUS0pPNmVGaVdtb lVta1BndUR4VTFNY2RqZ0VqVUo0W lJBNTIu', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bb'),
    username: 'gio',
    password: Binary.createFromBase64('JDJiJDEyJEMzNWphTEg2MldqV1ZkYS5yZ2dJaE9IMmwvVnNpdUpvYnZ6RVNzS1VobnZ3RFB1c jZodGnt', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bc'),
    username: 'GamerJob',
    password: Binary.createFromBase64('JDJiJDEyJDVuM2p iCHV0U3pZr0NXM lMxZnpsdGVTNFJ1U3hXODhgZ2trdUUS0EQxN3NHdVkyQ lZwd2Vh', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bd'),
    username: 'raisen',
    password: Binary.createFromBase64('JDJiJDEyJHh lmkJQW lvsZ jZrMVR iTS9JVWdvde9ZMWJyZ1AuSXdhaGVoQm5sZGRKa jhwaGhtOVV4amFx', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133be'),
    username: 'flint',
    password: Binary.createFromBase64('JDJiJDEyJFZ6Lk1rM jhZL2FzUEN3cWZOdGRPV0h1azBSck lZrM l0UEVqR l lZcXp5SGxMM25kNkxWZTlZ', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133bf'),
    username: 'luna',
    password: Binary.createFromBase64('JDJiJDEyJE8zN1J6ZVV5Qy9NVExocG8vW lE4Z3o2bHVuG1aTzZDVFJ5QU1TYmR3OG9Ob1B6ekRNa3lX', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133c0'),
    username: 'kai',
    password: Binary.createFromBase64('JDJiJDEyJG9FbFZoWnR0dzN0dU9YYzFtVkxKT3Y1dTFadVV2aFBmUFJzUGFBTVg5V01KMHRON3dxbmJu', 0)
  },
  {
    _id: ObjectId('6817a063c7cd9bc1ce0133c1'),
    username: 'val',
    password: Binary.createFromBase64('JDJiJDEyJHzyb0ViVVEXLzVYQzhTT0lJbW l5SEtXdzR4T3VzbUFMVXZ2UnpZcFVYaGh0aGV5V3k3OGdZ', 0)
  }
]
game>

```

(({parameter},{attribute: 1}))

```

game> db.items.find({stat: "atk"}, {bonus: 1})
[
  { _id: ObjectId('6817a308c7cd9bc1ce0133d6'), bonus: 5 },
  { _id: ObjectId('6817a308c7cd9bc1ce0133d7'), bonus: 10 },
  { _id: ObjectId('6817a308c7cd9bc1ce0133d8'), bonus: 20 }
]

```

({}, {attributes})

```
game> db.items.find({}, {_id: 0, name: 1})
[
  { name: 'Iron Blade' },
  { name: 'Steel Sword' },
  { name: 'Flame Sabre' },
  { name: 'Light Shoes' },
  { name: 'Swift Boots' },
  { name: 'Phantom Greaves' },
  { name: 'Leather Armor' },
  { name: 'Chainmail' },
  { name: 'Dragon Plate' },
  { name: 'Minor Health Potion' },
  { name: 'Major Health Potion' },
  { name: 'Elixir of Vitality' }
]
```

({parameter},{attribute: 0})

```
game> db.items.find({name: "Elixir of Vitality"}, {_id: 0})
[
  {
    name: 'Elixir of Vitality',
    type: 'potion',
    stat: 'hp',
    tier: 3,
    bonus: 20,
    cost: 250
  }
]
```

Using Five Operators

Utilizing Operators in the Source Code

```
owned_items = items.find({"_id": {"$in": owned_item_ids}})
shopItems = items.find({"_id": {"$nin": owned_item_ids}})
characters.update_one({"_id": ObjectId(_id)}, {"$addToSet": {"items": ObjectId(itemId)}})
characters.update_one({"_id": ObjectId(char_id)}, {"$inc": {"stats." + stat: points }})
characters.update_one({"_id": ObjectId(char_id)}, {"$set": {"points" : score}})
```

Mongosh Samples

Using \$in

```
game> db.items.find({_id: { $in: [ObjectId('6817a308c7cd9bc1ce0133e1'), ObjectId('6817a308c7cd9bc1ce0133e0')] } })
[
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133e0'),
    name: 'Major Health Potion',
    type: 'potion',
    stat: 'hp',
    tier: 2,
    bonus: 10,
    cost: 120
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133e1'),
    name: 'Elixir of Vitality',
    type: 'potion',
    stat: 'hp',
    tier: 3,
    bonus: 20,
    cost: 250
  }
]
game>
```

Using \$nin

```
game> db.items.find({_id: { $nin: [ObjectId('6817a308c7cd9bc1ce0133e1'),
...
... ObjectId('6817a308c7cd9bc1ce0133e0'), ObjectId('6817a308c7cd9bc1ce0133df'),
... ObjectId('6817a308c7cd9bc1ce0133de'), ObjectId('6817a308c7cd9bc1ce0133dd'),
... ObjectId('6817a308c7cd9bc1ce0133dc'), ObjectId('6817a308c7cd9bc1ce0133db')] } })
...
[
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d6'),
    name: 'Iron Blade',
    type: 'weapon',
    stat: 'atk',
    tier: 1,
    bonus: 5,
    cost: 100
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d7'),
    name: 'Steel Sword',
    type: 'weapon',
    stat: 'atk',
    tier: 2,
    bonus: 10,
    cost: 250
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d8'),
    name: 'Flame Sabre',
    type: 'weapon',
    stat: 'atk',
    tier: 3,
    bonus: 20,
    cost: 500
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133d9'),
    name: 'Light Shoes',
    type: 'armor',
    stat: 'spd',
    tier: 1,
    bonus: 3,
    cost: 80
  },
  {
    _id: ObjectId('6817a308c7cd9bc1ce0133da'),
    name: 'Swift Boots',
    type: 'armor',
    stat: 'spd',
    tier: 2,

```

Using \$addToSet

```
game> db.characters.updateOne({_id: ObjectId('6817a1e7c7cd9bc1ce0133cc')}, {$addToSet: {items: ObjectId('6817a308c7cd9bc1ce0133da')}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
game> db.characters.find({_id: ObjectId('6817a1e7c7cd9bc1ce0133cc')})
[
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133cc'),
    user_id: ObjectId('6817830339595684e701337b'),
    characterName: 'nash1',
    stats: { attack: 5, speed: 2, defense: 3 },
    items: [ ObjectId('6817a308c7cd9bc1ce0133da') ],
    points: 100,
    gold: 50
  }
]
```

Using \$inc

```
game> db.characters.updateOne({_id: ObjectId('6817a1e7c7cd9bc1ce0133cc')}, {$inc: {"stats.attack": 2}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
game> db.characters.find({_id: ObjectId('6817a1e7c7cd9bc1ce0133cc')})
[
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133cc'),
    user_id: ObjectId('6817830339595684e701337b'),
    characterName: 'nash1',
    stats: { attack: 7, speed: 2, defense: 3 },
    items: [ ObjectId('6817a308c7cd9bc1ce0133da') ],
    points: 100,
    gold: 50
  }
]
```

Using \$set

```
game> db.characters.updateOne({_id: ObjectId('6817a1e7c7cd9bc1ce0133cc')}, {$set: {points: 999}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
game> db.characters.find({_id: ObjectId('6817a1e7c7cd9bc1ce0133cc')})
[
  {
    _id: ObjectId('6817a1e7c7cd9bc1ce0133cc'),
    user_id: ObjectId('6817830339595684e701337b'),
    characterName: 'nash1',
    stats: { attack: 7, speed: 2, defense: 3 },
    items: [ ObjectId('6817a308c7cd9bc1ce0133da') ],
    points: 999,
    gold: 50
  }
]
```

References:

- Adentunji, D. (2024). *How Databases Guarantee Isolation – Pessimistic vs Optimistic Concurrency Control Explained*. <https://www.freecodecamp.org/news/how-databases-guarantee-isolation/>
- Amazon Web Services (n. d.). *Types of NoSQL databases*. <https://docs.aws.amazon.com/whitepapers/latest/choosing-an-aws-nosql-database/types-of-nosql-databases.html>
- ayusghoe (2017). *Fight for 200 commits*. <https://github.com/ayushgoel/system-design>
- Bloria S. (2024). *Evolution and Impact of NoSQL Databases: A Comprehensive Review*. International Research Journal of Modernization in Engineering and Technology and Science Volume:06/Issue:06/June-2024
- Brooker, M. (2023). *NoSQL: The Baby and the Bathwater*. <https://brooker.co.za/blog/2023/01/30/nosql.html>
- Fierro G., Chhatrapati S. (2021). *CS 186, Fall 2021, Course Notes*. Retrieved from: <https://cs186berkeley.net/resources/static/notes/n17-NoSQL.pdf>
- Hare, N. B. (2017). *Development and deployment of multiplayer online games: GDD, authoritative servers, communications/'No Bugs' Hare*. ITHare. com Website GmbH.
- Jonkey A., Gomstyn A. (2025). *Structured vs unstructured data: What's the difference?* Retrieved from: <https://www.ibm.com/think/topics/structured-vs-unstructured-data>
- Lau W., (2024). *The CAP Theorem, ACID vs. BASE*. IEMS5730/IERG4330/ESTR4316 Spring 2024
- Stearns, R. E., & Rosenkrantz, D. J. (1981, April). Distributed database concurrency controls using before-values. In Proceedings of the 1981 ACM SIGMOD international conference on Management of data (pp. 74-83).

Appendix

code structure

/api

auth.lua

manageUser.lua

misc.lua

/gameplay

wordState.lua

/images

Background.png

/sounds

/states

atkTraining.lua

buffStats.lua

charSelect.lua

createNewCharacter.lua

defTraining.lua

fight.lua

index.lua

inventory.lua

login.lua

playMenu.lua

register.lua

shop.lua

speedTraining.lua

statsHelp.lua

alert.lua

app.py

button.lua

buttonManager.lua

classic.lua

config.lua

hidden.lua

json.lua

main.lua

print_r.lua

session_manager.lua

text.lua

timer.lua

words.lua

app.py

```
from flask import Flask, request, jsonify

from pymongo import MongoClient

from bson.objectid import ObjectId

import bcrypt


app = Flask(__name__)


client = MongoClient("mongodb://localhost:27017/")

db = client["gameSample"] # create or connect to a database

account = db["accounts"] # creating or accessing a collection

characters = db["characters"]

items = db["items"]


# Sample Delete

# account.delete_many({})


@app.route('/login', methods = ['POST'])

def login():

    if request.method == 'POST':

        data = request.form

        username = data.get('username')

        password = data.get('password', "")


        user = account.find_one({"username": username})
```

```
print(user)
```

```
if user and bcrypt.checkpw(password.encode(), user['password']):
```

```
    user_id = str(user['_id'])
```

```
    return jsonify({"status": "success", "user_id": user_id}), 200
```

```
return "failed", 401
```

```
@app.route('/checkChars', methods = ['POST'])
```

```
def checkChars():
```

```
    if request.method == 'POST':
```

```
        data = request.form
```

```
        _id = data.get('_id')
```

```
        characterCount = characters.count_documents({"user_id": ObjectId(_id)})
```

```
        if characterCount > 0:
```

```
            return jsonify ({"characterCount": characterCount}), 200
```

```
        else:
```

```
            return "noChar", 201
```

```
return "failed", 401
```

```
@app.route('/getCharacter', methods = ['POST'])
```

```
def getCharacter():
```

```

if request.method == 'POST':

    data = request.get_json()

    print(data)

    _id = data.get('_id')

    allUserChars = characters.find({"_id": ObjectId(_id)})

    result = []

    for character in allUserChars:

        character['_id'] = str(character['_id'])

        character['user_id'] = str(character['user_id'])

        if isinstance(character.get("items"), list):

            character["items"] = [str(item) for item in character["items"]]

        result.append(character)

    return jsonify(result), 200

return jsonify({"status" : "failed"}), 401

# Read

@app.route('/getChars', methods = ['POST'])

def getChars():

```

```

if request.method == 'POST':

    data = request.get_json()

    print(data)

    _id = data.get('_id')

    allUserChars = characters.find({"user_id": ObjectId(_id)})

    result = []

    for character in allUserChars:

        character['_id'] = str(character['_id'])

        character['user_id'] = str(character['user_id'])

        if isinstance(character.get("items"), list):

            character["items"] = [str(item) for item in character["items"]]

        result.append(character)

    print(result)

    return jsonify(result), 200

    return jsonify({"status" : "failed"}), 401

```

Update

```
@app.route('/modifyStat', methods = ['POST'])
```

```
def modifyStat():
```

```

if request.method == 'POST':

    data = request.get_json()

    _id = data.get('_id')

    char_id = data.get('char_id')

    stat = data.get('stat')

    points = data.get('points')

    characters.update_one({ "_id": ObjectId(char_id) }, {"$inc": { "stats." + stat: points } })

    return jsonify({"status": "success"}), 200

return jsonify({"status" : "failed"}), 401

@app.route('/addChar', methods = ['POST'])
def addChar():

    if request.method == 'POST':

        data = request.form

        _id = data.get('_id')

        charname = data.get('charname')

        insertedCharacter = characters.insert_one({"user_id": ObjectId(_id), "characterName" : charname,
"stats": {"attack" : 1, "speed": 1, "defense": 1}, "items": [], "points": 0, "gold": 0})

        print(insertedCharacter)

    if insertedCharacter:

```

```
return "success", 200
```

```
return "failed", 401
```

```
# Create
```

```
@app.route('/register', methods = ['POST'])
```

```
def register():
```

```
    if request.method == 'POST':
```

```
        data = request.form
```

```
        username= data.get('username')
```

```
        password = data.get('password', "")
```

```
        password_hashed = bcrypt.hashpw(password.encode(),bcrypt.gensalt())
```

```
        if account.find_one({"username" : username}):
```

```
            return "exists", 200
```

```
        # start transaction
```

```
        result = account.insert_one({"username": username, "password": password_hashed})
```

```
        print(result)
```

```
        return "success", 200
```

```
@app.route('/setHighScore', methods = ['POST'])
```

```
def setHighScore():
```

```

if request.method == 'POST':

    data = request.get_json()

    print("Raw data:", request.data)

    print("Parsed JSON:", data)

    char_id = data.get("char_id")

    score = data.get("score")

    char_score = characters.find_one({"_id": ObjectId(char_id)}, {"_id": 0, "points": 1})

    print(char_id)

    print(char_score)

    if not char_score or char_score['points'] < score:

        characters.update_one({"_id": ObjectId(char_id)}, {"$set": {"points" : score}})

    return jsonify({"status": "success"}), 200

return jsonify({"status" : "failed"}), 401

@app.route('/getShopItems', methods=['POST'])
def getShopItems():

    data = request.get_json()

    char_id = data.get('char_id')

    owned_item_ids = []

```

```

character = characters.find_one({"_id": ObjectId(char_id)})

if character:

    owned_item_ids = character.get('items', [])

owned_item_ids = [ObjectId(item_id) for item_id in owned_item_ids]

shopItems = items.find({"_id": {"$nin": owned_item_ids}})

result = []

for shopItem in shopItems:

    shopItem['_id'] = str(shopItem['_id'])

    shopItem['name'] = str(shopItem['name'])

    shopItem['type'] = str(shopItem['type'])

    shopItem['stat'] = str(shopItem['stat'])

    shopItem['tier'] = str(shopItem['tier'])

    shopItem['bonus'] = str(shopItem['bonus'])

    shopItem['cost'] = str(shopItem['cost'])

    result.append(shopItem)

return jsonify(result), 200

@app.route('/addGold', methods = ['POST'])

def addGold():

    if request.method == 'POST':

```

```
data = request.get_json()
```

```
_id = data.get('_id')
```

```
gold = data.get('gold')
```

```
characters.update_one({ "_id": ObjectId(_id) }, {"$inc": { "gold" : gold} })
```

```
return jsonify({"status": "success"}), 200
```

```
return jsonify({"status" : "failed"}), 401
```

```
@app.route('/reduceGold', methods = ['POST'])
```

```
def reduceGold():
```

```
    if request.method == 'POST':
```

```
        data = request.get_json()
```

```
        print("Data: ")
```

```
        print(data)
```

```
        _id = data.get('_id')
```

```
        gold = data.get('gold')
```

```
        gold = -1 * int(gold)
```

```
        characters.update_one({ "_id": ObjectId(_id) }, {"$inc": { "gold" : gold} })
```

```
return jsonify({"status": "success"}), 200
```

```
return jsonify({"status" : "failed"}), 401
```

```
@app.route('/addItem', methods = ['POST'])
```

```
def addItem():
```

```
    if request.method == 'POST':
```

```
        data = request.get_json()
```

```
        _id = data.get('_id')
```

```
        itemId = data.get('itemId')
```

```
        characters.update_one({"_id": ObjectId(_id)}, {"$addToSet": {"items": ObjectId(itemId)}})
```

```
        return jsonify({"status": "success"}), 200
```

```
    return jsonify({"status" : "failed"}), 401
```

```
@app.route('/getItems', methods=['POST'])
```

```
def getItems():
```

```
    data = request.get_json()
```

```
    char_id = data.get('char_id')
```

```
    owned_item_ids = []
```

```
    character = characters.find_one({"_id": ObjectId(char_id)})
```

```
if character:
```

```
    owned_item_ids = character.get('items', [])
```

```
owned_item_ids = [ObjectId(item_id) for item_id in owned_item_ids]
```

```
owned_items = items.find({"_id": {"$in": owned_item_ids}})
```

```
result = []
```

```
for item in owned_items:
```

```
    result.append({
```

```
        '_id': str(item['_id']),
```

```
        'name': str(item['name']),
```

```
        'type': str(item['type']),
```

```
        'stat': str(item['stat']),
```

```
        'tier': str(item['tier']),
```

```
        'bonus': str(item['bonus']),
```

```
        'cost': str(item['cost'])
```

```
    })
```

```
return jsonify(result), 200
```

alert.lua

```
alert = Button:extend()
```

```
local bigFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 30)
```

```
local smallFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 15)
```

```
function alert:setColor(r, g, b)
```

```
    self.r = r
```

```
    self.g = g
```

```
    self.b = b
```

```
end
```

```
function alert:draw()
```

```
    love.graphics.setColor(self.r, self.g, self.b)
```

```
    love.graphics.rectangle("fill", self.x, self.y, self.width, self.height)
```

```
    love.graphics.setFont(bigFont)
```

```
    love.graphics.setColor(0, 0, 0)
```

```
    love.graphics.print(self.text, self.textX, self.textY)
```

```
end
```

```
return alert
```

button.lua

```
Button = Object:extend(Object)
```

```
local bigFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 30)
```

```
local smallFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 15)
```

```
function Button:new(numButtons, orderNum, width, height, text, label)
```

```
    local totalWhitespace = love.graphics.getHeight() - (height * numButtons)
```

```
    local whitespace = totalWhitespace / (numButtons + 1)
```

```
    local whitespaceInstance = whitespace * orderNum
```

```
    local blockspace = height * (orderNum - 1)
```

```
    self.x = love.graphics.getWidth() / 2 - (width / 2)
```

```
    self.y = whitespaceInstance + blockspace
```

```
    self.width = width
```

```
    self.height = height
```

```
    self.text = text
```

```
    self.textX = (love.graphics.getWidth() / 2) - (bigFont:getWidth(text) / 2)
```

```
    self.textY = whitespaceInstance + blockspace + ((height / 2) - (bigFont:getHeight() / 2))
```

```
    self.selected = false
```

```
    self.labelText = label
```

```
end
```

```
function Button:update()
```

```
    self.textX = (love.graphics.getWidth() / 2) - (bigFont:getWidth(self.text) / 2)
```

```
end
```

```
function Button:printValues()
```

```
    print(self.x)
```

```
    print(self.y)
```

```

        print(self.width)

        print(self.height)

    end

function Button:draw()

    love.graphics.setColor(255, 255, 255)

    if self.selected == true then

        love.graphics.rectangle("fill", self.x, self.y, self.width, self.height)

        love.graphics.setFont(bigFont)

        love.graphics.setColor(0, 0, 0)

        love.graphics.print("> " .. self.text, self.textX - bigFont:getWidth("> "), self.textY)

    else

        love.graphics.rectangle("line", self.x, self.y, self.width, self.height)

        love.graphics.setFont(bigFont)

        love.graphics.print(self.text, self.textX, self.textY)

    end

    love.graphics.setFont(smallFont)

    love.graphics.setColor(255, 255, 255)

    love.graphics.print(self.labelText, self.x, self.y - smallFont:getHeight())

end

return Button

```

buttonManager.lua

```
local buttonManager = {}
```

```
local pointer = 1
```

```
local totalNum = 1
```

```
local buttons = {}
```

```
local config = require("config")
```

```
function buttonManager.setUp(tableOfButtons, amount)
```

```
    buttons = tableOfButtons
```

```
    totalNum = amount
```

```
    pointer = 1
```

```
    buttons[pointer].selected = true
```

```
end
```

```
function buttonManager.down()
```

```
    buttons[pointer].selected = false
```

```
    if not (pointer == totalNum) then
```

```
        pointer = pointer + 1
```

```
    end
```

```
    buttons[pointer].selected = true
```

```
    config.sounds.minimal[1]:clone():play()
```

```
end
```

```
function buttonManager.up()
    buttons[pointer].selected = false

    if not (pointer == 1) then
        pointer = pointer - 1
    end

    buttons[pointer].selected = true
    config.sounds.minimal[2]:clone():play()
end

return buttonManager
```

classic.lua

```
--  
  
-- classic  
  
--  
  
-- Copyright (c) 2014, rxi  
  
--  
  
-- This module is free software; you can redistribute it and/or modify it under  
-- the terms of the MIT license. See LICENSE for details.  
  
--  
  
  
local Object = {}  
  
Object.__index = Object  
  
  
function Object:new() end  
  
  
function Object:extend()  
    local cls = {}  
    for k, v in pairs(self) do  
        if k:find("__") == 1 then  
            cls[k] = v  
        end  
    end  
    end  
  
    cls.__index = cls  
  
    cls.super = self  
  
    setmetatable(cls, self)
```

```
        return cls
    end

function Object:implement(...)
    for _, cls in pairs({ ... }) do
        for k, v in pairs(cls) do
            if self[k] == nil and type(v) == "function" then
                self[k] = v
            end
        end
    end
end

end
```

```
function Object:is(T)
    local mt = getmetatable(self)
    while mt do
        if mt == T then
            return true
        end
        mt = getmetatable(mt)
    end
    return false
end
```

```
function Object:__tostring()
```

```
        return "Object"
    end

    function Object:__call(...)
        local obj = setmetatable({}, self)
        obj:new(...)
        return obj
    end

    return Object
```

conf.lua

```
function love.conf(t)

    t.window.title = "Game Sample"

end
```

config.lua

```
local config = {}

local x = 1

config.fonts = {

    big = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 30),
    medium = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 20),
    small = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 15),
}

config.sounds = {

    mainMusic = love.audio.newSource("sounds/little-slimex27s-adventure-151007.mp3",
"stream"),

    minimal = {

        love.audio.newSource("sounds/ui_sounds/Minimalist10.ogg", "static"),
        love.audio.newSource("sounds/ui_sounds/Minimalist11.ogg", "static"),
        love.audio.newSource("sounds/ui_sounds/Minimalist12.ogg", "static"),
        love.audio.newSource("sounds/ui_sounds/Minimalist13.ogg", "static"),

    },

    type = {
```

```

        love.audio.newSource("sounds/ui_sounds/Retro2.ogg", "static"),
        love.audio.newSource("sounds/ui_sounds/Retro3.ogg", "static"),
        love.audio.newSource("sounds/ui_sounds/Retro4.ogg", "static"),
        love.audio.newSource("sounds/ui_sounds/Retro5.ogg", "static"),
    },

    enter = love.audio.newSource("sounds/ui_sounds/Retro9.ogg", "static"),

    back = love.audio.newSource("sounds/ui_sounds/Wood Block2.ogg", "static"),

    slay = love.audio.newSource("sounds/slay.mp3", "static"),

    beep = love.audio.newSource("sounds/beep.mp3", "static"),

}

config.images = {

    menuBackground = love.graphics.newImage("images/Background.png"),

}

function config.typing()

    config.sounds.type[x]:clone():play()

    x = x + 1

    if x == 5 then

        x = 1

    end

end

return config

```

hidden.lua

```
hidden = Button:extend()
```

```
local bigFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 30)
```

```
local smallFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 15)
```

```
function hidden:draw()
```

```
    love.graphics.setColor(255, 255, 255)
```

```
    if self.selected == true then
```

```
        love.graphics.rectangle("fill", self.x, self.y, self.width, self.height)
```

```
        love.graphics.setFont(bigFont)
```

```
        love.graphics.setColor(0, 0, 0)
```

```
        love.graphics.print(
```

```
            "> " .. string.rep(" ", string.len(self.text)),
```

```
            self.textX - bigFont:getWidth("> "),
```

```
            self.textY
```

```
        )
```

```
    else
```

```
        love.graphics.rectangle("line", self.x, self.y, self.width, self.height)
```

```
        love.graphics.setFont(bigFont)
```

```
        love.graphics.print(string.rep(" ", string.len(self.text)), self.textX, self.textY)
```

```
    end
```

```
    love.graphics.setFont(smallFont)
```

```
    love.graphics.setColor(255, 255, 255)
```

```
    love.graphics.print(self.labelText, self.x, self.y - smallFont:getHeight())
```

end

return hidden

main.lua

local currentState

local config = require("config")

local session = require("session_manager")

local function switchState(stateName)

 local states = {

 menu = require("states.index"),

 register = require("states.register"),

 login = require("states.login"),

 playMenu = require("states.playMenu"),

 buffStats = require("states.buffStats"),

 charSelect = require("states.charSelect"),

 createNewCharacter = require("states.createNewCharacter"),

 atkTraining = require("states.atkTraining"),

 speedTraining = require("states.speedTraining"),

 defenseTraining = require("states.defTraining"),

 statsHelp = require("states.statsHelp"),

 fight = require("states.fight"),

 shop = require("states.shop"),

```
        inventory = require("states.inventory"),
    }

    currentState = states[stateName]

    currentState.load()
end
```

```
function love.load()

    math.randomseed(os.time())

    Object = require("classic")
    local Button = require("button")
    local hidden = require("hidden")
    local alert = require("alert")
    local timer = require("timer")

    switchState("menu")

    config.sounds.mainMusic:setVolume(0.3)
    config.sounds.mainMusic:setLooping(true)
    config.sounds.mainMusic:play()

    love.keyboard.setKeyRepeat(true)
end
```

```
function love.textinput(t)
```

```
    if currentState.textinput then
```

```
        currentState.textinput(t)
```

```
    end
```

```
end
```

```
function love.keypressed(key, scancode, isrepeat)
```

```
    if currentState.keypressed then
```

```
        currentState.keypressed(key, scancode, isrepeat, switchState, session)
```

```
    end
```

```
end
```

```
function love.update(dt)
```

```
    if currentState.update then
```

```
        currentState.update(dt)
```

```
    end
```

```
end
```

```
function love.draw()
```

```
    love.graphics.setColor(255, 255, 255, 255)
```

```
    if currentState.draw then
```

```
        currentState.draw()
```

```
    end
```

```
end
```

text.lua

```
local text = {}
```

```
function text.printtext(textvalue, position_x, position_y)
```

```
    local x, y
```

```
    if position_x == 1 then
```

```
        x = 0
```

```
    elseif position_x == 2 then
```

```
        x = (love.graphics.getWidth() / 2) - (love.graphics.getFont():getWidth(textvalue) / 2)
```

```
    elseif position_x == 3 then
```

```
        x = (love.graphics.getWidth()) - love.graphics.getFont():getWidth(textvalue)
```

```
    end
```

```
    if position_y == 1 then
```

```
        y = 0
```

```
    elseif position_y == 2 then
```

```
        y = (love.graphics.getHeight() / 2) - (love.graphics.getFont():getHeight() / 2)
```

```
    elseif position_y == 3 then
```

```
        y = (love.graphics.getHeight()) - love.graphics.getFont():getHeight()
```

```
    end
```

```
    love.graphics.print(textvalue, x, y)
```

```
end
```

```
return text
```

```
timer.lua
```

```
Timer = Object.extend(Object)
```

```
local bigFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 30)
```

```
local smallFont = love.graphics.newFont("DepartureMonoNerdFontMono-Regular.otf", 15)
```

```
function Timer:new(y, time)
```

```
    local width = 400
```

```
    self.x = love.graphics.getWidth() / 2 - (width / 2)
```

```
    self.y = y
```

```
    self.width = width
```

```
    self.fixedWidth = width
```

```
    self.height = 30
```

```
    self.time = time
```

```
    self.elapsed = 0
```

```
    self.finished = false
```

```
end
```

```
function Timer:update(dt)
```

```
    self.elapsed = math.min(self.elapsed + dt, self.time)
```

```
    local progress = self.elapsed / self.time
```

```
    self.width = self.fixedWidth * (1 - progress)
```

```

        if self.elapsed == self.time then
            self.finished = true
        end
    end
end

function Timer:draw()
    love.graphics.setColor(255, 255, 255)
    love.graphics.rectangle("fill", self.x, self.y, self.width, self.height)
end

return Timer

```

words.lua

```

local json = require("json") -- or "lunajson"

local contents = love.filesystem.read("words_by_length.json")
local word_data = json.decode(contents)

local words = {}

-- Function to get a random word of a given length
function words.get_random(length)
    local key = tostring(length)

```

```
        local list = word_data[key]

        if not list then

            return nil

        end

        local index = math.random(1, #list)

        return list[index]

    end

end
```

```
return words
```

auth.lua

```
-- src/api/auth.lua
```

```
local http = require("socket.http")
```

```
local ltn12 = require("ltn12")
```

```
local json = require("json")
```

```
local auth = {}
```

```
-- optional: move this helper here if only used for auth
```

```
local function url_encode(str)
```

```
    str = string.gsub(str, "[^%w]", function(c)
```

```
        return string.format("%%%02X", string.byte(c))
```

```
    end)
```

```
    return str
```

```
end
```

```

function auth.login(username, password)

    local encodedUsername = url_encode(username)

    local encodedPassword = url_encode(password)

    local hashedPassword = love.data.encode("string", "hex", love.data.hash("sha256",
encodedPassword))

    local request_body = "username=" .. encodedUsername .. "&password=" .. hashedPassword


    local response_body = {}

    local res, code, _, _ = http.request({

        url = "http://127.0.0.1:5000/login",

        method = "POST",

        headers = {

            ["Content-Type"] = "application/x-www-form-urlencoded",

            ["Content-Length"] = tostring(#request_body),

        },

        source = ltn12.source.string(request_body),

        sink = ltn12.sink.table(response_body),

    })


    local responseText = table.concat(response_body)


    return responseText, code

end

```

```

function auth.register(username, password)

    local encodedUsername = url_encode(username)

```

```

    local encodedPassword = url_encode(password)

    local hashedPassword = love.data.encode("string", "hex", love.data.hash("sha256",
encodedPassword))

    local request_body = "username=" .. encodedUsername .. "&password=" .. hashedPassword


    local response_body = {}

    local res, code, _, _ = http.request({
        url = "http://127.0.0.1:5000/register",
        method = "POST",
        headers = {
            ["Content-Type"] = "application/x-www-form-urlencoded",
            ["Content-Length"] = tostring(#request_body),
        },
        source = ltn12.source.string(request_body),
        sink = ltn12.sink.table(response_body),
    })

    local responseText = table.concat(response_body)

    return responseText, code

end

return auth

```

manageUser.lua

```
-- src/api/auth.lua

local http = require("socket.http")

local ltn12 = require("ltn12")

local json = require("json")

local manageUser = {}

-- optional: move this helper here if only used for auth
local function url_encode(str)
    str = string.gsub(str, "[^%w]", function(c)
        return string.format("%%%02X", string.byte(c))
    end)
    return str
end

function manageUser.getCharacter(char_id)
    local request_body = json.encode({ ["_id"] = char_id })
    local response_body = {}

    local res, code, _, _ = http.request({
        url = "http://127.0.0.1:5000/getCharacter",
        method = "POST",
        headers = {
```

```

        ["Content-Type"] = "application/json",
        ["Content-Length"] = tostring(#request_body),
    },
    source = ltn12.source.string(request_body),
    sink = ltn12.sink.table(response_body),
})

local raw_response = table.concat(response_body)

print("Raw response from server:")

print(raw_response)

local responseText = json.decode(table.concat(response_body))

return responseText, code

end

function manageUser.setHighScore(char_id, score)

    local request_body = json.encode({ char_id = tostring(char_id), score = score })

    print(json.decode(request_body))

    local response_body = {}

    local res, code, _, _ = http.request({

        url = "http://127.0.0.1:5000/setHighScore",

        method = "POST",

        headers = {

            ["Content-Type"] = "application/json",

```

```

        ["Content-Length"] = tostring(#request_body),
    },
    source = ltn12.source.string(request_body),
    sink = ltn12.sink.table(response_body),
})

local raw_response = table.concat(response_body)

print("Raw response from server:")

print(raw_response)

local responseText = json.decode(table.concat(response_body))

return responseText, code

end

function manageUser.getChars(user_id)

    local request_body = json.encode({ ["_id"] = user_id })

    local response_body = {}

    local res, code, _, _ = http.request({
        url = "http://127.0.0.1:5000/getChars",
        method = "POST",
        headers = {
            ["Content-Type"] = "application/json",
            ["Content-Length"] = tostring(#request_body),
        },
    },

```

```
        source = ltn12.source.string(request_body),  
        sink = ltn12.sink.table(response_body),  
    })
```

```
local raw_response = table.concat(response_body)  
  
print("Raw response from server:")  
  
print(raw_response)
```

```
local responseText = json.decode(table.concat(response_body))  
  
return responseText, code
```

```
end
```

```
function manageUser.modifyStat(user_id, char_id, stat, points)
```

```
    local request_body = json.encode({ _id = user_id, char_id = char_id, stat = stat, points = points })
```

```
    local response_body = {}
```

```
    local res, code, _, _ = http.request({  
        url = "http://127.0.0.1:5000/modifyStat",  
        method = "POST",  
        headers = {  
            ["Content-Type"] = "application/json",  
            ["Content-Length"] = tostring(#request_body),  
        },  
        source = ltn12.source.string(request_body),  
        sink = ltn12.sink.table(response_body),
```

```
}}
```

```
local raw_response = table.concat(response_body)
```

```
print("Raw response from server:")
```

```
print(raw_response)
```

```
local responseText = json.decode(table.concat(response_body))
```

```
return responseText, code
```

```
end
```

```
function manageUser.checkChars(user_id)
```

```
    local request_body = "_id=" .. user_id
```

```
    local response_body = {}
```

```
    local res, code, _, _ = http.request({
```

```
        url = "http://127.0.0.1:5000/checkChars",
```

```
        method = "POST",
```

```
        headers = {
```

```
            ["Content-Type"] = "application/x-www-form-urlencoded",
```

```
            ["Content-Length"] = tostring(#request_body),
```

```
        },
```

```
        source = ltn12.source.string(request_body),
```

```
        sink = ltn12.sink.table(response_body),
```

```
    })
```

```
    local responseText = table.concat(response_body)

    return responseText, code

end
```

```
function manageUser.addChar(user_id, charName)

    local request_body = "_id=" .. user_id .. "&charname=" .. charName

    local response_body = {}

    local res, code, _, _ = http.request({

        url = "http://127.0.0.1:5000/addChar",

        method = "POST",

        headers = {

            ["Content-Type"] = "application/x-www-form-urlencoded",

            ["Content-Length"] = tostring(#request_body),

        },

        source = ltn12.source.string(request_body),

        sink = ltn12.sink.table(response_body),

    })

    local responseText = table.concat(response_body)

    return responseText, code

end
```

```
function manageUser.addGold(char_id, gold_amount)

    local request_body = json.encode({ _id = char_id, gold = gold_amount })
```

```

local response_body = {}

local res, code, _, _ = http.request({
    url = "http://127.0.0.1:5000/addGold",
    method = "POST",
    headers = {
        ["Content-Type"] = "application/json",
        ["Content-Length"] = tostring(#request_body),
    },
    source = ltn12.source.string(request_body),
    sink = ltn12.sink.table(response_body),
})

local raw_response = table.concat(response_body)

print("Raw response from server:")

print(raw_response)

local responseText = json.decode(table.concat(response_body))

return responseText, code

end

function manageUser.reduceGold(char_id, gold_amount)

    local request_body = json.encode({ _id = char_id, gold = gold_amount })

    local response_body = {}

```

```

local res, code, _, _ = http.request({
    url = "http://127.0.0.1:5000/reduceGold",
    method = "POST",
    headers = {
        ["Content-Type"] = "application/json",
        ["Content-Length"] = tostring(#request_body),
    },
    source = ltn12.source.string(request_body),
    sink = ltn12.sink.table(response_body),
})

local raw_response = table.concat(response_body)

print("Raw response from server:")

print(raw_response)

local responseText = json.decode(table.concat(response_body))

return responseText, code

end

function manageUser.addItem(char_id, itemId)

    local request_body = json.encode({ _id = char_id, itemId = itemId })

    local response_body = {}

    local res, code, _, _ = http.request({
        url = "http://127.0.0.1:5000/addItem",

```

```

        method = "POST",

        headers = {

            ["Content-Type"] = "application/json",

            ["Content-Length"] = tostring(#request_body),

        },

        source = ltn12.source.string(request_body),

        sink = ltn12.sink.table(response_body),

    })

    local raw_response = table.concat(response_body)

    print("Raw response from server:")

    print(raw_response)

    local responseText = json.decode(table.concat(response_body))

    return responseText, code

end

return manageUser

```

misc.lua

```
-- src/api/auth.lua

local http = require("socket.http")

local ltn12 = require("ltn12")

local json = require("json")

local misc = {}

function misc.getShopItems(char_id)

    local request_body = json.encode({ char_id = char_id })

    local response_body = {}

    local res, code, _, _ = http.request({

        url = "http://127.0.0.1:5000/getShopItems",

        method = "POST",

        headers = {

            ["Content-Type"] = "application/json",

            ["Content-Length"] = tostring(#request_body),

        },

        source = ltn12.source.string(request_body),

        sink = ltn12.sink.table(response_body),

    })

    local raw_response = table.concat(response_body)
```

```
print("Raw response from server:")

print(raw_response)


local responseText = json.decode(table.concat(response_body))

return responseText, code

end
```

```
function misc.getItems(char_id)

    local request_body = json.encode({ char_id = char_id })

    local response_body = {}


    local res, code, _, _ = http.request({

        url = "http://127.0.0.1:5000/getItems",

        method = "POST",

        headers = {

            ["Content-Type"] = "application/json",

            ["Content-Length"] = tostring(#request_body),

        },

        source = ltn12.source.string(request_body),

        sink = ltn12.sink.table(response_body),

    })


    local raw_response = table.concat(response_body)

    print("Raw response from server:")

    print(raw_response)
```

```
        local responseText = json.decode(table.concat(response_body))

        return responseText, code
    end
```

```
return misc
```

wordState.lua

```
local wordManager = {}

local words = require("words")

local config = require("config")

local word = ""

local pointer = 0

local completed = false

function wordManager.getState()

    return completed
end
```

```
local function get_length_range(length_label)

    local ranges = {

        short = { 2, 4 },

        medium = { 5, 7 },

        long = { 8, 10 },
```

```
        longest = { 11, 14 },  
    }  
end
```

```
local range = ranges[length_label]  
  
if not range then  
    return nil -- or error("Invalid length label")  
end
```

```
    return math.random(range[1], range[2])  
end
```

```
function wordManager.loadWord(length)  
    word = words.get_random(get_length_range(length))  
    pointer = 0  
  
    completed = false  
end
```

```
function wordManager.input(text)  
    print(text)  
  
    if text == string.sub(word, pointer + 1, pointer + 1) then  
        pointer = pointer + 1  
        config.typing()  
    else  
        config.sounds.back:clone():play()  
    end  
end
```

end

print("Pointer Value: " .. pointer)

print("String word: " .. string.len(word))

if pointer == string.len(word) then

 completed = true

end

end

function wordManager.draw()

 love.graphics.setFont(config.fonts.big)

 love.graphics.setColor(0, 0, 0)

 love.graphics.rectangle("fill", 0, love.graphics.getHeight() / 2 - 50, love.graphics.getWidth(), 100)

 love.graphics.setColor(255, 255, 255, 0.5)

 love.graphics.print(

 word,

 love.graphics.getWidth() / 2 - config.fonts.big:getWidth(word) / 2,

 (love.graphics.getHeight() / 2) - config.fonts.big:getHeight() / 2

)

 love.graphics.setColor(255, 255, 255, 0.5)

 love.graphics.print(

```

        string.sub(word, 1, pointer),

        love.graphics.getWidth() / 2 - config.fonts.big:getWidth(word) / 2,

        (love.graphics.getHeight() / 2) - config.fonts.big:getHeight() / 2

    )
end

return wordManager

```

atkTraining.lua

```

local state = {}

local manageUser = require("api.manageUser")

local buttonManager = require("buttonManager")

local config = require("config")

local alert = require("alert")

local wordManager = require("gameplay.wordState")

local session = require("session_manager")

local counter, total

local alertLevel, alertLevelBool

require("print_r")

local function fibonacci(n)

    if n <= 0 then

        return 0

    end

```

```

    if n == 1 then
        return 1
    end

    local a, b = 0, 1

    for i = 2, n do
        a, b = b, a + b
    end

    return b
end

function state.load()

    wordManager.loadWord("long")

    counter = 0

    total = fibonacci(session["currentChar"]["stats"]["attack"])

    print("total")

    print(total)

    alertLevel = alert(1, 1, 500, 100, "Level Up!", "")

    alertLevel:setColor(0, 255, 0)

    alertLevelBool = false

end

function state.textinput(t)

    if alertLevelBool == true then

```

```

        alertLevelBool = false

        return

    end

    print(t)

    wordManager.input(t)

    print("Word Manage State: ")

    print(wordManager.completed)

    if wordManager.getState() then

        wordManager.loadWord("long")

        counter = counter + 1

        if counter == total then

            local response, code = manageUser.modifyStat(session["user_id"],
session["currentChar"]["id"], "attack", 1)

            if response["status"] == "success" then

                print("status success")

                local response_2, code_2 =
manageUser.getCharacter(session["currentChar"]["id"])

                session["currentChar"]["stats"] = response_2[1]["stats"]

                alertLevelBool = true

            end

```

```
        counter = 0

        total = fibonacci(session["currentChar"]["stats"]["attack"])

    end

end

end
```

```
function state.keypressed(key, scancode, isrepeat, switchState, session)
```

```
    if alertLevelBool == true then
```

```
        alertLevelBool = false
```

```
        return
```

```
    end
```

```
    if scancode == "escape" then
```

```
        switchState("buffStats")
```

```
    end
```

```
end
```

```
function state.update(dt) end
```

```
function state.draw()
```

```
    love.graphics.draw(config.images.menuBackground, 0, -150)
```

```
    love.graphics.setFont(config.fonts.small)
```

```
    love.graphics.setFont(config.fonts.big)
```

```
    love.graphics.print(counter .. "/" .. total)
```

```
wordManager.draw()

if alertLevelBool then
    alertLevel:draw()
end

end

return state
```

buffStats.lua

```
local state = {}

local attackButton, speedButton, defenseButton, helpButton

local config = require("config")

local buttonManager = require("buttonManager")

function state.load()

    attackButton = Button(4, 1, 500, 100, "Finger Muscles (+Atk)", "")
    speedButton = Button(4, 2, 500, 100, "Finger Agility (+Spd)", "")
    defenseButton = Button(4, 3, 500, 100, "Finger Callus (+Def)", "")
    helpButton = Button(4, 4, 500, 100, "Help", "")

    buttonManager.setUp({ attackButton, speedButton, defenseButton, helpButton }, 4)

end
```

```
function state.keypressed(key, scancode, isrepeat, switchState, user_id)
```

```
    if scancode == "up" then
```

```
        buttonManager.up()
```

```
    elseif scancode == "down" then
```

```
        buttonManager.down()
```

```
    elseif scancode == "space" or scancode == "return" then
```

```
        if attackButton.selected then
```

```
            switchState("atkTraining")
```

```
        elseif speedButton.selected then
```

```
            switchState("speedTraining")
```

```
        elseif defenseButton.selected then
```

```
            switchState("defenseTraining")
```

```
        elseif helpButton.selected then
```

```
            switchState("statsHelp")
```

```
        end
```

```
    elseif scancode == "escape" then
```

```
        switchState("playMenu")
```

```
    end
```

```
end
```

```
function state.draw()
```

```
    love.graphics.draw(config.images.menuBackground, 0, -150)
```

```
    love.graphics.setFont(config.fonts.small)
```

```
    love.graphics.setFont(config.fonts.big)
```

```
        attackButton:draw()

        speedButton:draw()

        defenseButton:draw()

        helpButton:draw()

end
```

```
return state
```

charSelect.lua

```
local state = {}

local session = require("session_manager")

local json = require("json")

local userCharacters = {}

local firstChar, secondChar, thirdChar

local buttonManager = require("buttonManager")

local config = require("config")

local manageUser = require("api.manageUser")

local charState = { 0, 0, 0 }

--if code == 200 then

--user_id = json.decode(responseText)["user_id"]

--send

function state.load()
```

```
local responseText, code = manageUser.checkChars(session["user_id"])

local count, allChars, codeChars

charState = { 0, 0, 0 }

print(session)

print(session["user_id"])

print(code)

if code == 201 then

    firstChar = Button(3, 1, 500, 100, "New Character", "")

    secondChar = Button(3, 2, 500, 100, "New Character", "")

    thirdChar = Button(3, 3, 500, 100, "New Character", "")

else

    count = json.decode(responseText)["characterCount"]

    allChars, codeChars = manageUser.getChars(session["user_id"])

    if count == 1 then

        firstChar = Button(3, 1, 500, 100, allChars[1]["characterName"], "")

        secondChar = Button(3, 2, 500, 100, "New Character", "")

        thirdChar = Button(3, 3, 500, 100, "New Character", "")

        session["firstChar"] = {

            characterName = allChars[1]["characterName"],

            id = allChars[1]["_id"],
```

```
stats = allChars[1]["stats"],  
items = allChars[1]["items"],  
points = allChars[1]["points"],  
gold = allChars[1]["gold"],  
}
```

```
charState[1] = 1
```

```
elseif count == 2 then
```

```
firstChar = Button(3, 1, 500, 100, allChars[1]["characterName"], "")  
secondChar = Button(3, 2, 500, 100, allChars[2]["characterName"], "")  
thirdChar = Button(3, 3, 500, 100, "New Character", "")
```

```
session["firstChar"] = {  
    characterName = allChars[1]["characterName"],  
    id = allChars[1]["_id"],  
    stats = allChars[1]["stats"],  
    items = allChars[1]["items"],  
    points = allChars[1]["points"],  
    gold = allChars[1]["gold"],  
}
```

```
session["secondChar"] = {  
    characterName = allChars[2]["characterName"],  
    id = allChars[2]["_id"],  
    stats = allChars[2]["stats"],
```

```
        items = allChars[2]["items"],  
        points = allChars[2]["points"],  
        gold = allChars[2]["gold"],  
    }
```

```
charState[1] = 1
```

```
charState[1] = 1
```

```
charState[2] = 1
```

```
elseif count == 3 then
```

```
    firstChar = Button(3, 1, 500, 100, allChars[1]["characterName"], "")
```

```
    secondChar = Button(3, 2, 500, 100, allChars[2]["characterName"], "")
```

```
    thirdChar = Button(3, 3, 500, 100, allChars[3]["characterName"], "")
```

```
session["firstChar"] = {
```

```
    characterName = allChars[1]["characterName"],
```

```
    id = allChars[1]["_id"],
```

```
    stats = allChars[1]["stats"],
```

```
    items = allChars[1]["items"],
```

```
    points = allChars[1]["points"],
```

```
    gold = allChars[1]["gold"],
```

```
}
```

```
session["secondChar"] = {
```

```
    characterName = allChars[2]["characterName"],
```

```

        id = allChars[2]["_id"],
        stats = allChars[2]["stats"],
        items = allChars[2]["items"],
        points = allChars[2]["points"],
        gold = allChars[2]["gold"],
    }

    session["thirdChar"] = {
        characterName = allChars[3]["characterName"],
        id = allChars[3]["_id"],
        stats = allChars[3]["stats"],
        items = allChars[3]["items"],
        points = allChars[3]["points"],
        gold = allChars[3]["gold"],
    }

    charState[1] = 1
    charState[2] = 1
    charState[3] = 1
end

end

buttonManager.setUp({ firstChar, secondChar, thirdChar }, 3)

print("Status Code:", code)

```

```
    print("Response:", responseText)

end

function state.keypressed(key, scancode, isrepeat, switchState, user_id)

    if scancode == "up" then

        buttonManager.up()

    elseif scancode == "down" then

        buttonManager.down()

    elseif scancode == "space" or scancode == "return" then

        if firstChar.selected then

            if charState[1] == 1 then

                session["currentChar"] = session["firstChar"]

                switchState("playMenu")

            else

                switchState("createNewCharacter")

            end

        elseif secondChar.selected then

            if charState[2] == 1 then

                session["currentChar"] = session["secondChar"]

                switchState("playMenu")

            else

                switchState("createNewCharacter")

            end

        elseif thirdChar.selected then

            if charState[3] == 1 then
```

```
        session["currentChar"] = session["thirdChar"]

        switchState("playMenu")

    else

        switchState("createNewCharacter")

    end

end

elseif scancode == "escape" then

    switchState("menu")

end

end

function state.draw()

    love.graphics.draw(config.images.menuBackground, 0, -150)

    love.graphics.setFont(config.fonts.small)

    love.graphics.setFont(config.fonts.big)

    firstChar:draw()

    secondChar:draw()

    thirdChar:draw()

end

return state
```

createNewCharacter.lua

```
local state = {}

local manageUser = require("api.manageUser")

local buttonManager = require("buttonManager")

local config = require("config")

local charNameEntryField, charNameEnter

local alert = require("alert")

local characterCreated, characterCreatedErr, characterCreatedBool, characterCreatedErrBool

function state.load()

    print("this runs!")

    charNameEntryField = Button(2, 1, 500, 100, "", "Character Name")

    charNameEnter = Button(2, 2, 500, 100, "Enter", "")

    buttonManager.setUp({ charNameEntryField, charNameEnter }, 2)

    buttonManager["pointer"] = 1

    characterCreatedErr = alert(1, 1, 500, 100, "Error", "")

    characterCreatedErr:setColor(255, 0, 0)

    characterCreatedErrBool = false

    characterCreated = alert(1, 1, 500, 100, "Character Created!", "")

    characterCreated:setColor(0, 255, 0)

    characterCreatedBool = false
```

end

function state.textinput(t)

 if charNameEntryField.selected then

 config.typing()

 if string.len(charNameEntryField.text) < 20 then

 charNameEntryField.text = charNameEntryField.text .. t

 end

 end

end

function state.keypressed(key, scancode, isrepeat, switchState, session)

 print("Is Entry Field Selected ")

 print(charNameEntryField.selected)

 print("Is Enter Field Selected")

 print(charNameEnter.selected)

 if characterCreatedBool or characterCreatedErrBool then

 switchState("charSelect")

 else

 if scancode == "up" then

 buttonManager.up()

 print("Is Entry Field Selected (Up)")

 print(charNameEntryField.selected)

 print("Is Enter Field Selected (Up)")

 print(charNameEnter.selected)

```

elseif scandcode == "down" then

    buttonManager.down()

    print("Is Entry Field Selected (Down)")

    print(charNameEntryField.selected)

    print("Is Enter Field Selected (Down)")

    print(charNameEnter.selected)

elseif scandcode == "backspace" then

    if charNameEntryField.selected then

        config.sounds.back:clone():play()

        charNameEntryField.text = charNameEntryField.text:sub(1, -2)

    end

elseif scandcode == "escape" then

    if charNameEntryField.selected then

        config.sounds.back:clone():play()

        switchState("charSelect")

    end

elseif scandcode == "return" then

    if charNameEnter.selected then

        config.sounds.enter:clone():play()

        local response, code = manageUser.addChar(session["user_id"],
charNameEntryField.text)

        if response == "success" then

            characterCreatedBool = true

        else

            characterCreatedErrBool = true

```

```
        end
    end
end
end
end
```

```
function state.update(dt)
    charNameEntryField:update()
    charNameEnter:update()
end
```

```
function state.draw()
    love.graphics.draw(config.images.menuBackground, 0, -150)
    love.graphics.setFont(config.fonts.small)
    love.graphics.setFont(config.fonts.big)

    if characterCreatedBool then
        characterCreated:draw()
    elseif characterCreatedErrBool then
        characterCreatedErr:draw()
    end

    charNameEntryField:draw()
    charNameEnter:draw()
end
```

```
return state
```

defTraining.lua

```
local state = {}
```

```
local manageUser = require("api.manageUser")
```

```
local buttonManager = require("buttonManager")
```

```
local config = require("config")
```

```
local alert = require("alert")
```

```
local wordManager = require("gameplay.wordState")
```

```
local session = require("session_manager")
```

```
local counter, total
```

```
local alertLevel, alertLevelBool
```

```
require("print_r")
```

```
local function fibonacci(n)
```

```
    if n <= 0 then
```

```
        return 0
```

```
    end
```

```
    if n == 1 then
```

```
        return 1
```

```
    end
```

```
    local a, b = 0, 1
```

```
    for i = 2, n do
        a, b = b, a + b
    end
    return b
end
```

```
function state.load()

    wordManager.loadWord("longest")

    counter = 0

    total = fibonacci(session["currentChar"]["stats"]["defense"])

    alertLevel = alert(1, 1, 500, 100, "Level Up!", "")

    alertLevel:setColor(0, 255, 0)

    alertLevelBool = false

end
```

```
function state.textinput(t)

    if alertLevelBool == true then
        alertLevelBool = false
        return
    end

    print(t)

    wordManager.input(t)
```

```

print("Word Manage State: ")

print(wordManager.completed)

if wordManager.getState() then

    wordManager.loadWord("long")

    counter = counter + 1

    if counter == total then

        local response, code = manageUser.modifyStat(session["user_id"],
session["currentChar"]["id"], "defense", 1)

        if response["status"] == "success" then

            print("status success")

            local response_2, code_2 =
manageUser.getCharacter(session["currentChar"]["id"])

            session["currentChar"]["stats"] = response_2[1]["stats"]

            alertLevelBool = true

        end

        counter = 0

        total = fibonacci(session["currentChar"]["stats"]["defense"])

    end

end

end
end

```

```
function state.keypressed(key, scancode, isrepeat, switchState, session)
```

```
    if alertLevelBool == true then
```

```
        alertLevelBool = false
```

```
        return
```

```
    end
```

```
    if scancode == "escape" then
```

```
        switchState("buffStats")
```

```
    end
```

```
end
```

```
function state.update(dt) end
```

```
function state.draw()
```

```
    love.graphics.draw(config.images.menuBackground, 0, -150)
```

```
    love.graphics.setFont(config.fonts.small)
```

```
    love.graphics.setFont(config.fonts.big)
```

```
    love.graphics.print(counter .. "/" .. total)
```

```
    wordManager.draw()
```

```
    if alertLevelBool then
```

```
        alertLevel:draw()
```

```
    end
```

```
end
```

```
return state
```

fight.lua

```
local state = {}
```

```
local manageUser = require("api.manageUser")
```

```
local buttonManager = require("buttonManager")
```

```
local misc = require("api.misc")
```

```
local config = require("config")
```

```
local alert = require("alert")
```

```
local wordManager = require("gameplay.wordState")
```

```
local session = require("session_manager")
```

```
local timer = require("timer")
```

```
local counter, dmgToEnemy, enemyHp, enemyDmg, yourHealth, score, yourDmg, yourSpd, yourDef,  
setScore, totalHealth
```

```
local alertLevelBool, alertOver
```

```
local slaySound = config.sounds.slay:clone()
```

```
local typeTimer
```

```
local base_time = 5
```

```
local min_time = 0.5
```

```
local difficulty = 0.05
```

```
require("print_r")
```

```
local function fibonacci(n)
```

```
    if n <= 0 then
```

```
        return 0
```

```
    end
```

```
    if n == 1 then
```

```
        return 1
```

```
    end
```

```
    local a, b = 0, 1
```

```
    for i = 2, n do
```

```
        a, b = b, a + b
```

```
    end
```

```
    return b
```

```
end
```

```
local function drawTextBar(x, y, current, maximum, symbol)
```

```
    symbol = symbol or "|"
```

```
    local ratio = math.max(0, math.min(1, current / maximum))
```

```
    local totalSlots = 30
```

```
    local fillSlots = math.floor(ratio * totalSlots + 0.5)
```

```
    local emptySlots = totalSlots - fillSlots
```

```
    local bar = string.rep(symbol, fillSlots) .. string.rep(" ", emptySlots)
```

```
    love.graphics.print(bar, x, y)
```

end

local function calculateTime()

 local timePerWord = (base_time + (1 + (yourSpd * 0.05))) * math.exp(-difficulty * counter)

 timePerWord = math.max(min_time, timePerWord)

 return timePerWord

end

local function calculateEnemyHealth(enemiesDefeated)

 local startHealth = 5

 local targetHealth = 60

 local anchorEnemy = 40

 local growthRate = math.log(targetHealth / startHealth) / anchorEnemy

 return startHealth * math.exp(growthRate * enemiesDefeated)

end

local function calculateEnemyDamage(enemiesDefeated)

 local startDamage = 1

 local targetDamage = 10

 local anchorEnemy = 40

 local growthRate = math.log(targetDamage / startDamage) / anchorEnemy

 return startDamage * math.exp(growthRate * enemiesDefeated)

end

local function round(n)

 return math.floor(n + 0.5)

end

local function getEnemyDamage()

 local reduction = yourDef / (yourDef + 10)

 local reducedDamage = enemyDmg * (1 - reduction)

 return math.max(1, math.floor(reducedDamage + 0.5))

end

function state.load()

 wordManager.loadWord("short")

 counter = 0

 dmgToEnemy = calculateEnemyHealth(counter)

 yourHealth = 5

 totalHealth = 5

 yourDmg = session["currentChar"]["stats"]["attack"]

 yourSpd = session["currentChar"]["stats"]["speed"]

 yourDef = session["currentChar"]["stats"]["defense"]

 enemyHp = calculateEnemyHealth(counter)

 enemyDmg = calculateEnemyDamage(counter)

 score = 0

 alertOver = alert(1, 1, 500, 100, "Score: " .. score, "")

```
alertOver:setColor(139, 128, 0)

setScore = false

local ownedItems, code = misc.getItems(session["currentChar"]["id"])

if code == 200 and ownedItems then
    for _, item in ipairs(ownedItems) do
        local stat = item.stat -- "attack" | "speed" | "defense"

        local bonus = tonumber(item.bonus)

        if stat == "atk" then
            yourDmg = yourDmg + bonus
        elseif stat == "spd" then
            yourSpd = yourSpd + bonus
        elseif stat == "def" then
            yourDef = yourDef + bonus
        elseif stat == "hp" then
            yourHealth = yourHealth + bonus
            totalHealth = yourHealth
        end
    end
end

-- update session so future code sees the buffed values too
session["currentChar"]["stats"]["attack"] = yourDmg
session["currentChar"]["stats"]["speed"] = yourSpd
session["currentChar"]["stats"]["defense"] = yourDef

end
```

```

        typeTimer = timer(400, calculateTime())
end

function state.textinput(t)
    if alertLevelBool == true then
        alertLevelBool = false
        return
    end

    print(t)
    wordManager.input(t)

    print("Word Manage State: ")
    print(wordManager.completed)

    if wordManager.getState() then
        if counter < 5 then
            wordManager.loadWord("short")
        elseif counter < 15 then
            wordManager.loadWord("medium")
        elseif counter < 30 then
            wordManager.loadWord("long")
        else
            wordManager.loadWord("longest")
        end
    end
end

```

```
dmgToEnemy = dmgToEnemy - yourDmg
```

```
print(dmgToEnemy)
```

```
if dmgToEnemy <= 0 then
```

```
    slaySound:stop()
```

```
    slaySound:play()
```

```
    config.sounds.slay:clone():play()
```

```
    score = score + counter
```

```
    counter = counter + 1
```

```
    enemyHp = round(calculateEnemyHealth(counter))
```

```
    dmgToEnemy = enemyHp
```

```
    enemyDmg = round(calculateEnemyDamage(counter))
```

```
else
```

```
    config.sounds.enter:clone():play()
```

```
end
```

```
typeTimer = timer(400, calculateTime())
```

```
if counter == total then
```

```
    -- local response, code = manageUser.modifyStat(session["user_id"],  
session["currentChar"]["id"], "speed", 1)
```

```
    -- if response["status"] == "success" then
```

```

--      print("status success")
--
--      local response_2, code_2 =
manageUser.getCharacter(session["currentChar"]["id"])
--      session["currentChar"]["stats"] = response_2[1]["stats"]
--
--      alertLevelBool = true
-- end
--
-- counter = 0
-- total = fibonacci(session["currentChar"]["stats"]["speed"])
-- typeTimer = timer(20, calculateSetTime())
end
end
end

```

```

function state.keypressed(key, scandcode, isrepeat, switchState, session)

    if alertLevelBool == true then
        alertLevelBool = false
        return
    end

    if scandcode == "escape" then
        switchState("playMenu")
    end
end
end

```

```

function state.update(dt)
    typeTimer:update(dt)

    if yourHealth > 0 then
        if typeTimer.finished then
            yourHealth = yourHealth - getEnemyDamage()
            typeTimer = timer(400, calculateTime())
            config.sounds.beep:clone():play()

            if counter < 5 then
                wordManager.loadWord("short")
            elseif counter < 15 then
                wordManager.loadWord("medium")
            elseif counter < 30 then
                wordManager.loadWord("long")
            else
                wordManager.loadWord("longest")
            end
        end
    end

    else
        yourHealth = 0
        alertOver = alert(1, 1, 500, 100, "Score: " .. score, "")
        alertOver:setColor(139, 128, 0)
    end
end

```

```

        if not setScore then

            print("Current Character ID:")

            manageUser.setHighScore(session["currentChar"]["id"], score)

            manageUser.addGold(session["currentChar"]["id"], round(score / 10))

            setScore = true

        end

        session["currentChar"]["gold"] = session["currentChar"]["gold"] + round(score / 10)

    end

    if session["currentChar"]["points"] < score then
        session["currentChar"]["points"] = score
    end

end

end

end

function state.draw()

    love.graphics.draw(config.images.menuBackground, 0, -150)

    love.graphics.setFont(config.fonts.big)

    love.graphics.setColor(1, 1, 1, 1)

    love.graphics.print("Enemy (" .. (counter + 1) .. ") Damage: " .. enemyDmg, 10, 10)

    love.graphics.print("Total Score: " .. score, 10, 50)

    local screenW, screenH = love.graphics.getDimensions()

    local areaH = 80

```

```
local areaY = screenH - areaH
```

```
love.graphics.setColor(0, 0, 0, 0.7)
```

```
love.graphics.rectangle("fill", 0, areaY, screenW, areaH)
```

```
love.graphics.setFont(config.fonts.small)
```

```
love.graphics.setColor(1, 1, 1, 1)
```

```
love.graphics.print(("You:  %d / %d"):format(yourHealth, totalHealth), 10, areaY + 10)
```

```
love.graphics.setColor(255, 0, 0)
```

```
love.graphics.print(  
    ("Enemy (" .. (counter + 1) .. "): %d / %d"):format(dmgToEnemy, enemyHp),  
    screenW / 2 + 10,  
    areaY + 10  
)
```

```
love.graphics.setColor(1, 1, 1, 1)
```

```
love.graphics.setFont(config.fonts.small) -- same font, ensures fw is correct
```

```
drawTextBar(10, areaY + 40, yourHealth, totalHealth, "")
```

```
love.graphics.setColor(255, 0, 0)
```

```
drawTextBar(screenW / 2 + 10, areaY + 40, dmgToEnemy, enemyHp, "")
```

```
love.graphics.setFont(config.fonts.big)
```

```
wordManager.draw()
```

```
typeTimer:draw()
```

```
        if yourHealth < 1 then
            alertOver:draw()
        end
    end
end

return state
```

index.lua

```
local state = {}

local loginButton, registerButton

local config = require("config")

function state.load()

    local Button = require("button")

    loginButton = Button(2, 1, 300, 100, "Login", "")
    registerButton = Button(2, 2, 300, 100, "Register", "")

    loginButton.selected = true

    love.keyboard.setKeyRepeat(true)
end

function state.textinput(t) end
```

```

function state.keypressed(key, scancode, isrepeat, switchState, user_id)

    if scancode == "up" or scancode == "down" then -- move down

        if loginButton.selected == false then

            loginButton.selected = true

            registerButton.selected = false

            config.sounds.minimal[1]:clone():play()

        else

            loginButton.selected = false

            registerButton.selected = true

            config.sounds.minimal[2]:clone():play()

        end

    elseif scancode == "return" or scancode == "space" then

        if registerButton.selected == true then

            switchState("register")

            config.sounds.enter:clone():play()

        elseif loginButton.selected then

            switchState("login")

            config.sounds.enter:clone():play()

        end

    end

end

end

function state.draw()

    love.graphics.draw(config.images.menuBackground, 0, -(love.graphics.getHeight() / 4))

```

```
        love.graphics.setFont(config.fonts.big)

        loginButton:draw()

        registerButton:draw()

    end
```

```
return state
```

inventory.lua

```
local state = {}

local config = require("config")

local misc = require("api.misc")

local manageUser = require("api.manageUser")

local session = require("session_manager")

local itemsResponse, code, totalItemsNum, currentItem
```

```
local function tablelength(T)
```

```
    local count = 0

    for _ in pairs(T) do

        count = count + 1

    end

    return count
```

```
end
```

```
function state.load()
```

```
itemsResponse, code = misc.getItems(session["currentChar"]["id"])

totalItemsNum = tablelength(itemsResponse)

currentItem = 1

print_r(code)

end
```

```
function state.keypressed(key, scancode, isrepeat, switchState, user_id)

    if scancode == "up" then

        currentItem = currentItem - 1

        if currentItem < 1 then

            currentItem = totalItemsNum

        end

    elseif scancode == "down" then

        currentItem = currentItem + 1

        if currentItem > totalItemsNum then

            currentItem = 1

        end

    elseif scancode == "escape" then

        switchState("playMenu")

    end

end

end
```

```
function state.update(dt) end
```

```
function state.draw()
```

```
    love.graphics.draw(config.images.menuBackground, 0, -150)
```

```
    love.graphics.setColor(0, 0, 0)
```

```
    love.graphics.rectangle("fill", 0, 150, love.graphics.getWidth(), 120)
```

```
    love.graphics.setColor(255, 255, 255)
```

```
    if itemsResponse[currentItem] then
```

```
        love.graphics.print("Item : " .. itemsResponse[currentItem]["name"], 200, 175)
```

```
        love.graphics.print(
```

```
            "Increases " .. itemsResponse[currentItem]["stat"] .. " by " ..  
itemsResponse[currentItem]["bonus"],
```

```
            200,
```

```
            200
```

```
        )
```

```
    end
```

```
end
```

```
return state
```

login.lua

```
local state = {}
```

```
local backToMain, enterLogin
```

```
local loginUsernameField, loginPasswordField
```

```
local usernameExistsErr, usernameExistsErrBool, loginSuccess, loginSuccessBool
```

```
local loginTableButtons = {}
```

```
local loginPointer
```

```
local config = require("config")
```

```
local auth = require("api.auth")
```

```
local text = require("text")
```

```
local json = require("json")
```

```
function state.load()
```

```
    Object = require("classic")
```

```
    local Button = require("button")
```

```
    local hidden = require("hidden")
```

```
    local alert = require("alert")
```

```
    backToMain = Button(4, 1, 300, 100, "Back", "")
```

```
    backToMain.selected = true
```

```
    enterLogin = Button(4, 4, 300, 100, "Enter", "")
```

```
    loginUsernameField = Button(4, 2, 600, 100, "", "Username")
```

```
    loginPasswordField = hidden(4, 3, 600, 100, "", "Password")
```

```
    usernameExistsErr = alert(1, 1, 500, 100, "Failed!", "")
```

```
    usernameExistsErr:setColor(255, 0, 0)
```

```
    usernameExistsErrBool = false
```

```

loginSuccess = alert(1, 1, 500, 100, "Account created!", "")

loginSuccess:setColor(0, 255, 0)

loginSuccessBool = false


loginTableButtons = { backToMain, loginUsernameField, loginPasswordField, enterLogin }

loginPointer = 1

end


function state.textinput(t)

    if loginUsernameField.selected then

        config.typing()

        if string.len(loginUsernameField.text) < 20 then

            loginUsernameField.text = loginUsernameField.text .. t

        end

    elseif loginPasswordField.selected then

        config.typing()

        loginPasswordField.text = loginPasswordField.text .. t

    end

end


end


function state.keypressed(key, scancode, isrepeat, switchState, session)

    usernameExistsErrBool = false

    loginSuccessBool = false

    if scancode == "up" then

        loginTableButtons[loginPointer].selected = false

    end

end

```

```
if loginPointer == 1 then
    loginPointer = 4
else
    loginPointer = loginPointer - 1
end

loginTableButtons[loginPointer].selected = true
config.sounds.minimal[loginPointer]:clone():play()
elseif scandcode == "down" then
    loginTableButtons[loginPointer].selected = false

    if loginPointer == 4 then
        loginPointer = 1
    else
        loginPointer = loginPointer + 1
    end

    loginTableButtons[loginPointer].selected = true
    config.sounds.minimal[loginPointer]:clone():play()
elseif scandcode == "space" or scandcode == "return" then
    if backToMain.selected then
        switchState("menu")
        config.sounds.enter:clone():play()
    elseif enterLogin.selected then
```

```

        local responseText, code = auth.login(loginUsernameField.text,
loginPasswordField.text)

        print("Status Code:", code)

        print("Response:", responseText)


        if code == 200 then

            session["user_id"] = json.decode(responseText)["user_id"]

            switchState("charSelect")

        else

            usernameExistsErrBool = true

        end


        if responseText == "exists" then

            usernameExistsErrBool = true

        elseif responseText == "success" then

            loginSuccessBool = true

        end

    end

elseif scancode == "backspace" then

    if loginUsernameField.selected then

        config.sounds.back:clone():play()

        loginUsernameField.text = loginUsernameField.text:sub(1, -2)

    elseif loginPasswordField.selected then

        config.sounds.back:clone():play()

        loginPasswordField.text = loginPasswordField.text:sub(1, -2)

    end

end

```

```

        end
    end

    function state.update(dt)

        loginUsernameField:update()

        loginPasswordField:update()

    end

    function state.draw()

        love.graphics.draw(config.images.menuBackground, 0, -150)

        love.graphics.setFont(config.fonts.small)

        text.printtext("Login", 2, 1)

        love.graphics.setFont(config.fonts.big)

        backToMain:draw()

        loginUsernameField:draw()

        loginPasswordField:draw()

        enterLogin:draw()

        if usernameExistsErrBool == true then

            usernameExistsErr:draw()

        elseif loginSuccessBool then

            loginSuccess:draw()

        end

    end

end

```

```
return state
```

playMenu.lua

```
local state = {}
```

```
local logoutButton, buffStatsButton, fightButton, leaderboardButton, inventoryButton, shopButton,  
playMenuButtons, playMenuPointer
```

```
local config = require("config")
```

```
local session = require("session_manager")
```

```
local function down()
```

```
    if not (playMenuPointer == 5) then
```

```
        playMenuPointer = playMenuPointer + 1
```

```
    end
```

```
end
```

```
local function up()
```

```
    if not (playMenuPointer == 1) then
```

```
        playMenuPointer = playMenuPointer - 1
```

```
    end
```

```
end
```

```
function state.load()
```

```
    print(session)
```

```
    logoutButton = Button(5, 1, 320, 80, "Logout", "")
```

```
    buffStatsButton = Button(5, 2, 320, 80, "Upgrade Stats", "")
```

```
    fightButton = Button(5, 3, 320, 80, "Fight", "")
```

```
inventoryButton = Button(5, 4, 320, 80, "Inventory", "")
```

```
shopButton = Button(5, 5, 320, 80, "Shop", "")
```

```
playMenuButtons = { logoutButton, buffStatsButton, fightButton, inventoryButton, shopButton }
```

```
playMenuPointer = 1
```

```
logoutButton.selected = true
```

```
end
```

```
function state.keypressed(key, scandcode, isrepeat, switchState, user_id)
```

```
  if scandcode == "up" then
```

```
    playMenuButtons[playMenuPointer].selected = false
```

```
    up()
```

```
    playMenuButtons[playMenuPointer].selected = true
```

```
    config.sounds.minimal[1]:clone():play()
```

```
  elseif scandcode == "down" then
```

```
    playMenuButtons[playMenuPointer].selected = false
```

```
    down()
```

```
    playMenuButtons[playMenuPointer].selected = true
```

```
    config.sounds.minimal[2]:clone():play()
```

```
  elseif scandcode == "space" or scandcode == "return" then
```

```
    if logoutButton.selected then
```

```
      user_id = "blank"
```

```
      switchState("menu")
```

```
    elseif buffStatsButton.selected then
```

```

        switchState("buffStats")
    elseif fightButton.selected then
        switchState("fight")
    elseif inventoryButton.selected then
        switchState("inventory")
    elseif shopButton.selected then
        switchState("shop")
    end
end
end

function state.draw()

    love.graphics.draw(config.images.menuBackground, 0, -150)

    love.graphics.setFont(config.fonts.medium)

    love.graphics.rectangle("fill", 10, 10, 220, 190)

    love.graphics.setColor(0, 0, 0)

    love.graphics.print("Attack : " .. session["currentChar"]["stats"]["attack"], 15, 15)
    love.graphics.print("Speed  : " .. session["currentChar"]["stats"]["speed"], 15, 50)
    love.graphics.print("Defense : " .. session["currentChar"]["stats"]["defense"], 15, 85)
    love.graphics.print("Hi Score: " .. session["currentChar"]["points"], 15, 120)
    love.graphics.print("Gold   : " .. session["currentChar"]["gold"], 15, 155)

    logoutButton:draw()

    buffStatsButton:draw()

```

```
        fightButton:draw()

        inventoryButton:draw()

        shopButton:draw()

    end
```

```
return state
```

register.lua

```
local state = {}

local backToMain, enterRegister

local registerUsernameField, registerPasswordField

local usernameExistsErr, usernameExistsErrBool, registerSuccess, registerSuccessBool

local registerTableButtons = {}

local registerPointer

local config = require("config")

local auth = require("api.auth")

local text = require("text")

function state.load()

    Object = require("classic")

    local Button = require("button")

    local hidden = require("hidden")

    local alert = require("alert")
```

```

backToMain = Button(4, 1, 300, 100, "Back", "")

backToMain.selected = true

enterRegister = Button(4, 4, 300, 100, "Enter", "")


registerUsernameField = Button(4, 2, 600, 100, "", "Username")
registerPasswordField = hidden(4, 3, 600, 100, "", "Password")


usernameExistsErr = alert(1, 1, 500, 100, "Account already exists!", "")
usernameExistsErr:setColor(255, 0, 0)
usernameExistsErrBool = false


registerSuccess = alert(1, 1, 500, 100, "Account created!", "")
registerSuccess:setColor(0, 255, 0)
registerSuccessBool = false


registerTableButtons = { backToMain, registerUsernameField, registerPasswordField,
enterRegister }

registerPointer = 1

end

function state.textinput(t)
    if registerUsernameField.selected then
        config.typing()
        if string.len(registerUsernameField.text) < 20 then
            registerUsernameField.text = registerUsernameField.text .. t
        end
    end
end

```

```
elseif registerPasswordField.selected then
    config.typing()
    registerPasswordField.text = registerPasswordField.text .. t
end
end
```

```
function state.keypressed(key, scandcode, isrepeat, switchState)

    usernameExistsErrBool = false
    registerSuccessBool = false

    if scandcode == "up" then
        registerTableButtons[registerPointer].selected = false

        if registerPointer == 1 then
            registerPointer = 4
        else
            registerPointer = registerPointer - 1
        end

        registerTableButtons[registerPointer].selected = true
        config.sounds.minimal[registerPointer]:clone():play()
    elseif scandcode == "down" then
        registerTableButtons[registerPointer].selected = false

        if registerPointer == 4 then
            registerPointer = 1
        end
    end
end
```

```

else

    registerPointer = registerPointer + 1

end

registerTableButtons[registerPointer].selected = true

config.sounds.minimal[registerPointer]:clone():play()

elseif scandcode == "space" or scandcode == "return" then

    if backToMain.selected then

        switchState("menu")

        config.sounds.enter:clone():play()

    elseif enterRegister.selected then

        local responseText, code = auth.register(registerUsernameField.text,
registerPasswordField.text)

        print("Status Code:", code)

        print("Response:", responseText)

        if responseText == "exists" then

            usernameExistsErrBool = true

        elseif responseText == "success" then

            registerSuccessBool = true

        end

    end

end

elseif scandcode == "backspace" then

    if registerUsernameField.selected then

        config.sounds.back:clone():play()

        registerUsernameField.text = registerUsernameField.text:sub(1, -2)

```

```
        elseif registerPasswordField.selected then
            config.sounds.back:clone():play()
            registerPasswordField.text = registerPasswordField.text:sub(1, -2)
        end
    end
end
```

```
function state.update(dt)
    registerUsernameField:update()
    registerPasswordField:update()
end
```

```
function state.draw()
    love.graphics.draw(config.images.menuBackground, 0, -150)
    love.graphics.setFont(config.fonts.small)
    text.printtext("Register", 2, 1)
    love.graphics.setFont(config.fonts.big)

    backToMain:draw()
    registerUsernameField:draw()
    registerPasswordField:draw()
    enterRegister:draw()

    if usernameExistsErrBool == true then
        usernameExistsErr:draw()
    end
end
```

```
        elseif registerSuccessBool then
            registerSuccess:draw()
        end
    end
end
```

```
return state
```

shop.lua

```
local state = {}

local config = require("config")

local misc = require("api.misc")

local manageUser = require("api.manageUser")

local session = require("session_manager")

local itemsResponse, code, totalItemsNum, currentItem
```

```
local function tablelength(T)
```

```
    local count = 0

    for _ in pairs(T) do
        count = count + 1
    end

    return count
```

```
end
```

```
function state.load()
```

```

itemsResponse, code = misc.getShopItems(session["currentChar"]["id"])

totalItemsNum = tablelength(itemsResponse)

currentItem = 1

print_r(code)

end

function state.keypressed(key, scandcode, isrepeat, switchState, user_id)

    if scandcode == "up" then

        currentItem = currentItem - 1

        if currentItem < 1 then

            currentItem = totalItemsNum

        end

    elseif scandcode == "down" then

        currentItem = currentItem + 1

        if currentItem > totalItemsNum then

            currentItem = 1

        end

    elseif scandcode == "return" then

        print("hello")

        if session["currentChar"]["gold"] >= tonumber(itemsResponse[currentItem]["cost"])

            then

                print("hello2")

```

```
manageUser.addItem(session["currentChar"]["id"], itemsResponse[currentItem]
["_id"])
```

```
manageUser.reduceGold(session["currentChar"]["id"],
itemsResponse[currentItem]["cost"])
```

```
session["currentChar"]["gold"] = session["currentChar"]["gold"] -
itemsResponse[currentItem]["cost"]
```

```
end
```

```
elseif scancode == "escape" then
```

```
switchState("playMenu")
```

```
end
```

```
end
```

```
function state.update(dt) end
```

```
function state.draw()
```

```
love.graphics.draw(config.images.menuBackground, 0, -150)
```

```
love.graphics.setColor(0, 0, 0)
```

```
love.graphics.rectangle("fill", 0, 150, love.graphics.getWidth(), 175)
```

```
love.graphics.setColor(255, 255, 255)
```

```
if itemsResponse[currentItem] then
```

```
love.graphics.print("Item : " .. itemsResponse[currentItem]["name"], 200, 175)
```

```
love.graphics.print(
```

```
"Increases " .. itemsResponse[currentItem]["stat"] .. " by " ..
itemsResponse[currentItem]["bonus"],
```

```
200,
```

```
200
```

```

    )
    love.graphics.print("Gold: " .. itemsResponse[currentItem]["cost"], 200, 225)

    love.graphics.print("Press Enter to Buy", 200, 250)

    love.graphics.print("Your Gold: " .. session["currentChar"]["gold"], 200, 275)
end
end

return state

```

speedTraining.lua

```

local state = {}

local manageUser = require("api.manageUser")

local buttonManager = require("buttonManager")

local config = require("config")

local alert = require("alert")

local wordManager = require("gameplay.wordState")

local session = require("session_manager")

local timer = require("timer")

local counter, total

local alertLevel, alertLevelBool

local typeTimer

local base_time = 3

```

```
local min_time = 0.5
```

```
local difficulty = 0.15
```

```
require("print_r")
```

```
local function fibonacci(n)
```

```
    if n <= 0 then
```

```
        return 0
```

```
    end
```

```
    if n == 1 then
```

```
        return 1
```

```
    end
```

```
    local a, b = 0, 1
```

```
    for i = 2, n do
```

```
        a, b = b, a + b
```

```
    end
```

```
    return b
```

```
end
```

```
local function calculateSetTime()
```

```
    local wordCount = fibonacci(session["currentChar"]["stats"]["speed"])
```

```
    local timePerWord = base_time * math.exp(-difficulty * session["currentChar"]["stats"]  
["speed"])
```

```
    timePerWord = math.max(min_time, timePerWord)
```

```
    return timePerWord * wordCount
```

end

function state.load()

wordManager.loadWord("short")

counter = 0

total = fibonacci(session["currentChar"]["stats"]["speed"])

typeTimer = timer(10, calculateSetTime())

alertLevel = alert(1, 1, 500, 100, "Level Up!", "")

alertLevel:setColor(0, 255, 0)

alertLevelBool = false

end

function state.textinput(t)

if alertLevelBool == true then

alertLevelBool = false

return

end

print(t)

wordManager.input(t)

print("Word Manage State: ")

print(wordManager.completed)

```

    if wordManager.getState() then

        wordManager.loadWord("short")

        counter = counter + 1

        if counter == total then

            local response, code = manageUser.modifyStat(session["user_id"],
session["currentChar"]["id"], "speed", 1)

            if response["status"] == "success" then

                print("status success")

                local response_2, code_2 =
manageUser.getCharacter(session["currentChar"]["id"])

                session["currentChar"]["stats"] = response_2[1]["stats"]

                alertLevelBool = true

            end

            counter = 0

            total = fibonacci(session["currentChar"]["stats"]["speed"])

            typeTimer = timer(20, calculateSetTime())

        end

    end

end
end

```

```

function state.keypressed(key, scancode, isrepeat, switchState, session)

```

```

    if alertLevelBool == true then
        alertLevelBool = false
        return
    end

    if scancode == "escape" then
        switchState("buffStats")
    end
end

function state.update(dt)
    typeTimer:update(dt)

    if typeTimer.finished then
        counter = 0
        typeTimer = timer(20, calculateSetTime())
    end
end

function state.draw()
    love.graphics.draw(config.images.menuBackground, 0, -150)

    love.graphics.setFont(config.fonts.small)

    love.graphics.setFont(config.fonts.big)

    love.graphics.print(counter .. "/" .. total)

```

```
wordManager.draw()
```

```
typeTimer:draw()
```

```
if alertLevelBool then
```

```
    alertLevel:draw()
```

```
end
```

```
end
```

```
return state
```

statsHelp.lua

```
local state = {}
```

```
local config = require("config")
```

```
function state.load() end
```

```
function state.keypressed(key, scancode, isrepeat, switchState, user_id)
```

```
    if scancode == "escape" then
```

```
        switchState("buffStats")
```

```
    end
```

```
end
```

```
function state.update(dt) end
```

```
function state.draw()

    love.graphics.draw(config.images.menuBackground, 0, -150)

    love.graphics.setColor(0, 0, 0)

    love.graphics.rectangle("fill", 0, 190, love.graphics.getWidth(), 150)

    love.graphics.setColor(255, 255, 255)

    love.graphics.print("Attack increases damage to enemy HP", 200, 200)

    love.graphics.print("Speed increases time to type out words", 200, 250)

    love.graphics.print("Defense reduces damage taken", 200, 300)

end

return state
```